AI-Based Food Recognition with Nutrition-Aware Recommendations

A Report submitted

in partial fulfillment of the requirement

for the award of the Degree of

Bachelor of Technology

in

Computer Science and Engineering

by

Jaspreet Singh (2022BCSE019)

Inderpal Singh (2022BCSE037)

Under the guidance of

Dr. Lavanya Madhuri Bollipo

Supervisor

Dr. Insha Majeed

Co-Supervisor

Department of Computer Science and Engineering
National Institute of Technology Srinagar
Kashmir 190006, INDIA

June 2026

CERTIFICATE

This is to certify that the project report entitled **AI-Based Food Recognition with Nutrition-Aware Recommendations** submitted by **Jaspreet Singh (2022BCSE019) and Inderpal Singh (2022BCSE037)** to the Department of Computer Science and Engineering, NIT Srinagar, in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a bona fide record of project work carried out under the supervision of Dr. Lavanya Madhuri Bollipo and co-supervision of Dr. Insha Majeed.

The work presented in this report has been prepared as a final year project in the area of computer vision, full-stack web engineering, nutrition data integration, and recommendation systems. To the best of our knowledge, the work described here has not been submitted elsewhere for the award of any degree or diploma.

Dr. Lavanya Madhuri Bollipo
Supervisor
Department of Computer Science and Engineering
NIT Srinagar

Dr. Insha Majeed
Co-Supervisor
Department of Computer Science and Engineering
NIT Srinagar

STUDENT DECLARATION

We declare that this project report titled **AI-Based Food Recognition with Nutrition-Aware Recommendations** is submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering. The project is a record of original work carried out by us under the supervision of Dr. Lavanya Madhuri Bollipo and co-supervision of Dr. Insha Majeed.

The matter embodied in this project, in full or in part, has not been submitted to any other institution or university for the award of any degree or diploma. We also declare that the work submitted by us is original and that all external datasets, software libraries, documentation sources, and research references used in the project have been acknowledged appropriately.

Jaspreet Singh
2022BCSE019

Inderpal Singh
2022BCSE037

Srinagar, 190006
June 2026

ACKNOWLEDGEMENTS

We express our sincere gratitude to our project guide, Dr. Lavanya Madhuri Bollipo, and co-supervisor, Dr. Insha Majeed, for their guidance, review, and technical direction throughout this final year project. Their feedback was especially useful when the work moved from a broad idea into practical problems such as cleaning dataset labels, checking food-class images, correcting nutrition mappings, and connecting the model output to a usable web interface.

We are thankful to the Department of Computer Science and Engineering, National Institute of Technology Srinagar, for providing the academic environment and opportunity to carry out this work. We also acknowledge the open-source communities behind Python, FastAPI, Next.js, Ultralytics YOLO, PyTorch, and the scientific computing tools used in this project. The availability of these tools allowed us to spend more time on project-specific issues such as class normalization, nutrition validation, and interface behavior.

We also thank our families, friends, and peers for their support, feedback, and patience during the design, implementation, debugging, testing, and documentation phases of the work.

Jaspreet Singh
2022BCSE019

Inderpal Singh
2022BCSE037

ABSTRACT

Manual food logging is easy to ignore because it asks the user to repeat several small steps after every meal: identify each dish, estimate the quantity, search for a matching nutrition entry, and enter the remaining items on the plate. Indian meals make this harder because one image may contain rice, dal, chutney, curry, fried snacks, sweets, and drinks, often with regional names or household variations. This project addresses that problem through an image-based Indian food recognition prototype with nutrition-aware recommendations. A user uploads a food image, the detector identifies visible food items, the backend maps those labels to Indian nutrition records, and the frontend presents calories, macronutrients, serving controls, and dietary suggestions.

The implemented prototype contains a FastAPI backend, a Next.js frontend, a SQLite nutrition database built mainly from the Indian Nutrient Database, and a merged YOLO-format dataset with 72 canonical Indian food classes. Five source datasets were combined after resolving spelling differences, regional terms, and duplicate labels that could otherwise split the same food into separate detector outputs. Examples such as `idly` and `idli`, or `satham` and `rice`, were normalized before training. The final balanced export contains 48,693 image-label pairs across training, validation, and test splits. Visual verification sheets were prepared for all 72 classes, which helped confirm class coverage and also exposed a few annotation-level issues.

The final detector was trained using YOLO11s on the 72-class dataset. The consolidated training CSV in `merge_results/` recorded a best validation mAP@50 of 0.83379 and a best validation mAP@50:95 of 0.69226 during the 100-epoch run. The latest split-level image-evaluation plots in the same folder report all-class mAP@50 values of 0.776 for validation and 0.765 for the held-out test split. The nutrition layer maps all 72 dataset classes to database entries through explicit semantic mappings and verified supplemental rows before using fuzzy matching. This choice came from early testing, where name similarity alone sometimes selected nutritionally unsuitable records. The backend includes endpoints for image analysis, nutrition validation, macro calculation, goals, health conditions, and recommendation generation. The frontend supports upload, detection visualization, serving adjustment, macro tracking, and personalized guidance.

Contents

List of Figures

List of Tables

ABBREVIATIONS AND NOTATIONS

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BMR	Basal Metabolic Rate
CNN	Convolutional Neural Network
CSV	Comma-Separated Values
FYP	Final Year Project
HTTP	Hypertext Transfer Protocol
INDB	Indian Nutrient Database
IoU	Intersection over Union
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
NMS	Non-Maximum Suppression
ORM	Object Relational Mapping
REST	Representational State Transfer
SQL	Structured Query Language
UI	User Interface
YOLO	You Only Look Once

Notations

B	Bounding box represented as center coordinates, width, and height.
C	Set of canonical food classes in the merged dataset.
P	Predicted probability or confidence score produced by the detector.
IoU	Area overlap ratio between predicted and ground-truth bounding boxes.
M_c	Nutrition mapping selected for class c.
E	Energy value in kilocalories per 100 grams.

Chapter 1

INTRODUCTION

1.1 Overview

Food logging looks simple when it is described as a single task, but in daily use it becomes repetitive. The user has to remember the meal, estimate the quantity, search for a suitable food entry, and repeat the process for every item on the plate. Indian meals make this more complicated. A normal plate may include rice, dal, chutney, curry, a fried snack, and a sweet in small portions. The meal may be obvious to the person eating it, but most logging applications still expect each item to be searched and entered manually.

Computer vision can reduce part of this effort, but it does not solve nutrition logging by itself. Food does not have a fixed shape, lighting changes its appearance, and some items are partly hidden by bowls, plates, or other foods. Indian dishes add another challenge because several classes share ingredients, colour, and texture. For example, `aloo_gobi`, `dum_aloo`, and `aloo_masala` can look similar in a cropped image, while chutneys may appear only as small side portions. During implementation, it became clear that a confident visual label is not enough. The label also has to be connected to a nutrition record that makes sense for that dish.

This project implements a food recognition and nutrition-assistance prototype for Indian meals. The final application uses a YOLO object detector, a curated Indian food dataset, a SQLite nutrition database derived mainly from the Indian Nutrient Database, a FastAPI backend, and a Next.js frontend. After an image is uploaded, the backend detects visible food items, maps each detected class to a nutrition entry, calculates macronutrients, and prepares suggestions using the user's goal and selected health context.

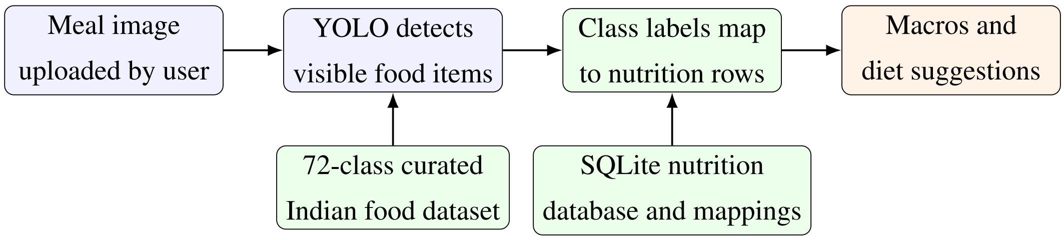


Figure 1.1: End-to-end workflow of the implemented food recognition and nutrition system

1.2 Motivation

The motivation came from two practical observations. First, food entry has to be quick enough for daily use. If logging takes too long, the user is likely to stop after a few days. Second, Indian food recognition needs labels that match the cuisine. A generic model may detect a plate, bowl, or snack-like region, but it will not reliably separate `rajma_curry`, `sambar`, `poha`, `thepla`, or `bhakarwadi`. The recommendation layer also needs context. The same meal can be reasonable for a muscle-gain goal and unsuitable for weight loss, while a carbohydrate-heavy meal should be explained differently for a diabetic user than for an endurance-focused user.

The second observation became clearer only after the first working version was tested. Detection and nutrition lookup are connected, but they are not the same problem. A detector may correctly return `samosa` or `palak_paneer`, but the application still has to connect that label to the correct database row. If this mapping is wrong, the interface may display calorie and macro values that look precise but are actually misleading. Early fuzzy-matching tests produced cases such as `palak_paneer` being linked to a generic cottage-cheese entry instead of spinach paneer. After that, nutrition mapping was handled as a core project task rather than a small final step.

1.3 Problem Statement

The problem addressed in this project is to design and implement a full-stack system that can:

1. Detect one or more Indian food items from an uploaded image.
2. Normalize detected class labels into stable canonical food names.
3. Map detected foods to nutrition database records with maximum safe coverage.

4. Estimate calories, protein, carbohydrates, and fat per detected item.

5. Provide personalized dietary recommendations using user goals and health conditions.

6. Present the results through an accessible web interface with bounding boxes, serving adjustment, and macro tracking.

The problem is therefore wider than training a detector. It includes dataset engineering, data validation, backend API design, frontend interaction design, and nutrition-aware application logic. Each part affects the final reliability. A clean frontend cannot fix a wrong food-to-nutrition mapping, and a trained detector becomes less useful if the class labels are inconsistent.

1.4 Objectives

The main objectives of the project are:

- To build a canonical Indian food dataset by merging multiple YOLO-format source datasets.
- To resolve duplicate and inconsistent food labels into a stable set of classes.
- To integrate a YOLO-based detection service into a FastAPI backend.
- To create a nutrition lookup service backed by a local SQLite database.
- To improve nutrition mapping quality through explicit class-to-database mappings.
- To implement macro calculation for different dietary goals and health profiles.
- To develop a Next.js frontend that supports image upload, detection visualization, nutrition cards, and recommendation display.
- To validate the dataset, class names, nutrition mappings, and service behavior.

1.5 Scope of the Project

The implemented prototype is limited to image-based recognition of Indian food classes present in the curated dataset. The model output is treated as a dish label, and nutrition values are reported per 100 grams unless the user changes the serving assumption in the frontend. The application does not estimate physical food volume or exact portion size from a single photograph. This boundary was kept intentionally because reliable portion estimation would require depth information, a known reference scale, segmentation, or controlled image capture. The prototype therefore focuses on a practical recognition-to-nutrition workflow that can be extended later.

The backend and frontend are designed for local full-stack deployment. The backend exposes REST-style endpoints for image analysis, macro calculation, validation, and recommendation generation. The frontend is built as a single-page workflow for uploading images and reviewing results. When configured, the recommendation module uses Groq-backed language model generation, but the numerical nutrition calculations remain inside the backend. This separation keeps recommendations from becoming the source of calorie or macro values.

1.6 Major Contributions

The major contributions are listed below. They are written from the completed implementation rather than only from the model-training stage, because the final output depends on the full pipeline.

1. A merged Indian food dataset containing 72 canonical classes and 48,693 exported image-label pairs.
2. A normalization pipeline that maps raw labels such as `AlooGobi`, `aloo gobhi`, `thengai chutney`, `satham`, and `medu vadai` into stable canonical labels.
3. Visual class verification artifacts showing real image samples for all 72 classes.
4. A nutrition database build process that imports INDB nutrition fields and creates a runtime SQLite database.
5. A safer nutrition mapping strategy that covers all 72 dataset classes using explicit semantic mappings and verified supplemental nutrition rows before fuzzy matching.
6. A FastAPI analysis endpoint that combines detection results, bounding boxes, nutrition data, and food-not-found handling.
7. A frontend interface that combines upload, detection visualization, serving adjustment, macro progress, and recommendations.

1.7 Report Organization

The report follows the same broad order as the project work. Chapter 2 reviews the supplied research papers and identifies the gap addressed by the implementation. Chapter 3 explains dataset construction, nutrition mapping, macro logic, and validation strategy. Chapter 4 presents the system design and implementation details in one combined chapter. Chapter 5 presents the experimental analysis, results, and discussion together so that the training evidence and interpretation remain in one place. Chapters 6 and 7 contain the conclusion and future work. The appendices include class mappings, API examples, and supporting verification material.

Chapter 2

LITERATURE REVIEW

2.1 Scope of the Review

This review uses the collected research papers as background for the implemented project. The papers cover food recognition, Indian cuisine datasets, nutrition databases, semantic mapping, and recommendation systems. They were not treated only as separate benchmarks. They were read with a practical question in mind: what is needed to move from an uploaded food image to a nutrition output that is usable in an application?

For this project, the papers fall into three useful groups. The first group discusses food localization and classification through YOLO, CNNs, transfer learning, segmentation, and compact recognition models. The second group deals with nutrition databases and the difficult task of connecting a visual food label to a food-composition row. The third group looks at recommendation systems and LLM-assisted meal planning, including the risk of treating generated nutrition estimates as exact values.

2.2 Expanded Paper Summaries

2.2.1 Vision and Object Detection

2.2.1.1 IndianFoodNet

IndianFoodNet is relevant because it treats Indian food recognition as an object-detection task instead of only an image-classification task [1]. YOLO-based bounding boxes are suitable for Indian meals, where chutney, dal, rice, fried snacks, and curries may appear together in one image. A single image label would usually miss some of these smaller or side items.

The work supported the use of a single-stage detector in the project architecture.

In this project, YOLO11s localizes food items first, and the backend then performs nutrition lookup for each detected class. IndianFoodNet mainly focuses on the visual detection stage. The present work continues after detection by adding database-backed macro values and recommendation output.

2.2.1.2 Khana Dataset

The Khana dataset paper addresses the shortage of large Indian cuisine benchmarks by presenting more than 131,000 images across 80 classes [2]. It also makes clear why Indian food recognition is difficult: several dishes share colour, texture, garnish, and serving style even when their ingredients and macro values are different.

This matched the problems seen while merging the project dataset. Canonical class normalization, train-only augmentation, and visual class verification were needed because a confused label affects more than detector accuracy. It can also select the wrong nutrition record later. The paper also shows why class-wise checking is useful, since aggregate metrics can hide weak classes.

2.2.1.3 Enhanced Food Image Recognition

The enhanced food image recognition study uses transfer learning with lightweight networks such as MobileNetV2 to keep inference cost low while maintaining acceptable precision [3]. In food logging, this is a practical requirement because users expect a result soon after taking or uploading a meal image.

The implemented application uses YOLO detection rather than single-image classification, but the same response-time concern remains. The backend has to receive the uploaded image, run inference, attach nutrition data, and return a response without feeling slow. For that reason, model size and service design were treated as deployment concerns, not only training choices.

2.2.1.4 Single-item Training for Multi-dish Recognition

The single-item training paper studies complex Indian platter recognition using a Squeeze-and-Excitation DenseNet121-style architecture with class-agnostic feature extraction [4]. The reported accuracy is high, but the pipeline is heavier and more complex to deploy in a simple web application.

The recognition path in this work is simpler. YOLO performs localization and classification in one inference pass, which made FastAPI integration more direct. This choice does not solve every problem caused by overlapping foods, but it was more practical for the prototype than building a segmentation-first pipeline.

2.2.1.5 Lightweight and Parameter-Optimized CNN

The lightweight CNN paper concentrates on latency reduction for real-time food calorie estimation [5]. It adjusts convolution, pooling, and dropout choices to make prediction faster while keeping accuracy usable. The main lesson for this project is simple: a meal-logging tool has to respond quickly enough for repeated use.

For this project, the paper supports the use of compact detector variants and a backend where image inference, nutrition lookup, and recommendation generation remain separate stages. Each stage can then be checked and improved independently. The main limitation from this project's viewpoint is that the food coverage is mostly generic rather than focused on Indian dishes.

2.2.1.6 What is in an Indian Thali?

The Indian Thali paper studies one of the harder cases for Indian food recognition: overlapping and partly hidden items in a thali meal [6]. It uses segmentation, pixel masking, and a multi-camera setup to separate items on the same plate.

The paper is useful because similar overlap also appears in ordinary meal photographs. However, its capture setup is not convenient for daily food logging. The implemented system works from a single uploaded image and uses YOLO bounding boxes instead of multi-camera segmentation. The boundaries are less precise, but the method fits the web-based prototype better.

2.2.2 Nutrition Databases and Semantic Mapping

2.2.2.1 Development of INDB

The Development of an Indian Food Composition Database paper is central to the nutrition part of this project [7]. It reports nutrient values for cooked Indian recipes, which are more useful for meal tracking than raw ingredient values alone. Cooking changes the final macro profile through oil absorption, water loss, ingredient ratios, and preparation method.

The backend converts the available INDB spreadsheet into a local SQLite database. This gives the application a fixed source for calories, protein, carbohydrates, and fat. The detector is used to identify the dish, while the database supplies the numerical macro values. This avoids asking a language model to guess nutrition values directly from an image.

2.2.2.2 Indian Food Composition Tables

The Indian Food Composition Tables provide a strong analytical baseline for raw Indian foods [8]. They are valuable because they come from standardized biochemical measurements and help explain ingredient-level nutrient profiles.

For this system, the limitation is that raw food values do not always match cooked meals. A curry, sweet, or fried snack can differ greatly from its raw ingredients after cooking. For this reason, the project uses cooked recipe-style INDB data as the main runtime source while still treating food-composition tables as an important reference.

2.2.2.3 Synergistic Frameworks for Indian Food Computing

The synergistic frameworks paper points out the mapping gap between visual food labels and nutrition database rows [9]. A detector may use one spelling, while the database may use a regional name or a longer recipe name. Without a mapping layer, even a correct visual label can fail during nutrition lookup.

The same issue appeared during implementation. Labels such as `satham`, `medu_vadai`, and `thengai_chutney` had to be normalized before they could be connected to database entries. The backend therefore uses canonical labels, explicit mappings, aliases, and conservative fuzzy matching instead of depending only on exact string equality.

2.2.2.4 Nutrition Information Estimation from Food Photos

The nutrition information estimation paper combines food or ingredient predictions from photographs with structured nutrition datasets [10]. This is close to the logic used here because nutrition is not treated as a direct pixel-to-calorie regression problem.

The prototype uses dish-level detections instead of ingredient-level mass estimation. This boundary was kept because exact food weight cannot be recovered reliably from one image without depth, scale, or controlled capture conditions. The interface therefore reports per-100g values and allows serving adjustment.

2.2.2.5 Label AI

Label AI maps barcode-scanned packaged foods to nutrition values and fitness goals [11]. That approach works well when the product already has a barcode and a standardized label. The identifier is clear, so nutrition lookup is comparatively direct.

Home-cooked Indian meals usually do not provide such identifiers. A plate of poha, rajma, idli, dosa, or thali has to be recognized visually. The present system replaces barcode input with YOLO detection and then connects the detected class to a SQLite nutrition database. This comparison explains why cooked-food applications need a visual-to-database mapping layer.

2.2.3 Recommendations, LLMs, and System Integration

2.2.3.1 Diet Engine

Diet Engine combines YOLOv8 food detection with an NLP chatbot for real-time dietary guidance [12]. It is one of the closer works to this project because it connects object detection with advice instead of stopping at classification.

The implemented workflow follows a similar broad loop but adapts it for Indian foods. The dataset, class names, and nutrition mappings are domain-specific, and the recommendation layer receives structured food and macro data from the backend. As a result, the advice is connected to the detected meal rather than only to typed user input.

2.2.3.2 AI-driven Personalized Meal Planning

The AI-driven personalized meal planning paper presents a web platform that uses generative AI for diets based on user goals and health constraints [13]. Its relevance is partly technical and partly practical, because a user needs one place to handle goals, health conditions, and dietary suggestions.

The implementation adopts the same full-stack view through a Next.js frontend and FastAPI backend. The difference is that the recommendation begins with an uploaded meal image. Detected foods and calculated macros are passed to the recommendation layer so that the advice is tied to what the user actually logged.

2.2.3.3 NutriGen

NutriGen shows that LLMs can generate personalized meal plans when they are constrained by nutrition databases and user preferences [14]. The useful idea is not only the use of an LLM, but the way structured nutrition context is provided to it.

This shaped the recommendation service design in the project. The backend calculates calories and macros first, then passes structured information to the recommendation layer. The LLM is therefore not used as the source of exact numerical nutrition values, which reduces unsupported calorie or macro claims.

2.2.3.4 LLMs and Computer Vision for Personalized Nutrition Advice

The LLM and computer vision nutrition paper combines OCR with language models to read printed food labels and generate dietary advice [15]. It follows a useful pattern: computer vision first extracts grounded information, and language generation is applied afterward.

The present system follows that pattern, but the grounded input is cooked-food detection instead of OCR. Most Indian meals do not have printed labels, so the meal image itself becomes the input. YOLO detections and database-backed macros provide the structured context for the recommendation service.

2.2.3.5 AI-Based Nutrition Recommendation System

The AI-based nutrition recommendation system uses combinatorial logic and user profile constraints to generate diet plans within macronutrient boundaries [16]. This is useful because it keeps recommendations within explicit nutrition limits instead of producing open-ended advice.

From this project's perspective, the limitation is that such a planner does not verify the meal the user actually ate. The implemented system adds a visual logging step before recommendations are produced, creating a clearer connection between the uploaded meal and the advice shown to the user.

2.2.3.6 Evaluation of ChatGPT for Nutrient Estimation

The ChatGPT nutrient-estimation evaluation reports that vision-language models can identify foods reasonably well but still struggle with exact macronutrient estimation from meal photographs [17]. The concern is that fluent text can make uncertain estimates appear more reliable than they are.

The implemented architecture avoids this problem by keeping numerical nutrition calculation outside the LLM. Calories, protein, carbohydrates, and fat come from the SQLite nutrition database and mapping table. The LLM receives already-computed values and is used only for explanation and recommendation text.

2.2.3.7 Visual Nutrition Analysis

The visual nutrition analysis study uses segmentation models such as UNet or DeepLab-style networks to isolate food pixels and estimate nutrition through regression [18]. Such an approach can work well when segmentation masks are accurate and the dataset contains detailed nutrition labels.

The implemented approach does not estimate nutrients directly from pixels. It separates recognition from nutrition lookup: YOLO identifies the food class, and the backend retrieves macros from a database. The design was easier to validate within the project scope, especially because exact serving-size estimation was not included.

2.3 Comparative Summary

Table 2.1 gives a short comparison of the reviewed papers. The summaries above explain how each paper influenced the implemented system.

Table 2.1: Condensed summary of reviewed research papers

S. No	Technique & Reference	About Technique	Applications	Dataset Used	Results	Limitations
1	YOLO architectures, IndianFood-Net [1]	Bounding-box detection.	Real-time Indian food tracking.	IFN: 5,500+ images.	YOLOv5 mAP 0.807; YOLOv8 0.671.	No nutrition database.
2	CNNs, ViT and ConvNeXT, Khana [2]	Fine-grained classification.	Indian cuisine benchmarking.	Khana: 131K images, 80 classes.	ConvNeXT-S top-1: 86.72%.	Single-label task.
3	Transfer learning, Enhanced Food CNNs [3]	MobileNetV2 retraining.	Fast web/mobile classification.	Custom data.	Weighted precision: 76%.	No multi-dish detection.
4	SE-DenseNet121, single-item training [4]	SE feature extraction.	Cluttered platter recognition.	30 Indian categories.	Top-1: 97.48%; saved 333–500 annotation hours.	Computationally heavy.
5	Lightweight CNN [5]	Tuned MaxPool and Dropout layers.	Mobile calorie estimation.	Food-101 and Fruit-360.	About 85%; 0.0013 s.	Mostly Western foods.
6	Object segmentation, Indian Thali [6]	Pixel masking and KNN matching.	Occluded thali segmentation.	ITD and WED.	Segmented 5–10 overlapping items.	Needs five-camera rig.
7	Biochemical aggregation, INDB [7]	Cooked recipe macro profiling.	Indian recipe nutrition.	INDB.	Macros for 1,014 cooked recipes.	Needs AI mapping bridge.
8	Analytical baseline, IFCT [8]	Direct lab measurements.	Raw Indian food reference.	NIN databank.	Standardized 528 foods.	Raw foods only.
9	Fuzzy semantic mapping [9]	Partial string matching to database rows.	Visual-to-database mapping.	Visual and nutrition tables.	Supports 72-class linking.	Needs clean taxonomy.
10	Pretrained CNNs and DB lookup [10]	Visual prediction plus nutrition lookup.	Multi-task macro extraction.	Yelp and Nutrition5k.	85% ingredient accuracy; F1 49.09%.	Multiple lookup overhead.
11	CatBoost and barcodes, Label AI [11]	Barcode APIs and ML scoring.	Packaged food tracking.	Open Food Facts.	R^2 0.9002; explained variance 0.9010.	Not for home-cooked meals.

S. No	Technique & Reference	About Technique	Applications	Dataset Used	Results	Limitations
12	YOLOv8 and NLP bot, Diet Engine [12]	Detection piped to chatbot.	Mobile diet consulting.	Custom: 6,692 images.	F1 0.86 at 0.264 confidence; mAP 0.897.	Weak Indian generalization.
13	LLM meal planning [13]	DeepSeek/ChatGPT diet generation.	Chronic disease diets.	User health profiles.	90% weight-loss accuracy; 87% diabetic alignment.	Manual inputs.
14	Prompted LLMs, NutriGen [14]	USDA-constrained generation.	Personalized plans.	USDA database.	Calorie error as low as 1.55%.	Requires manual logging.
15	OCR and LLM [15]	Printed label extraction for LLMs.	Context-aware diet chat.	Packaged labels.	100% text extraction for JSON output.	Cannot read cooked food pixels.
16	Combinatorial logic recommender [16]	Macro-boundary matching.	Seasonal diet generation.	Mediterranean database.	100% acceptable spring/summer plans for 840 users.	No visual verification.
17	VLM estimation, ChatGPT evaluation [17]	GPT-4V estimates macros from photos.	Zero-shot dietary assessment.	114 meal photos.	Mean correlation 0.62; exact match 33.3–57.9%.	Hallucinates exact math.
18	UNet segmentation, Visual Nutrition Analysis [18]	Segmentation and nutrient regression.	Direct macro estimation.	Nutrition5k.	Dice 97.69%; Jaccard 95.52%.	Hidden ingredients remain difficult.

2.4 Research Gap

The reviewed papers show that food recognition, nutrition lookup, and recommendation generation are often handled separately. Detection papers can localize foods, but they may not connect those labels to reliable nutrition records. Nutrition databases contain structured macro values, but they do not identify what is present in a user-uploaded image. Recommendation systems can generate diet suggestions, but they often depend on manual inputs and may not verify the meal visually.

The present work addresses that integration gap for Indian food. It combines YOLO-based detection, a canonical Indian food label set, SQLite-backed nutrition lookup, explicit class-to-database mapping, deterministic macro calculation, and structured recommendation generation. The workflow is broader than a detector-only system and avoids asking an LLM or vision-language model to guess exact nutrition values from a photograph.

Chapter 3

METHODOLOGY

3.1 Methodological Overview

The methodology follows the order in which the project became reliable during development. The dataset had to be cleaned first, because model integration is difficult to trust when the same food appears under different labels. The nutrition database also had to be checked before any recommendation output could be considered meaningful. Once these two parts were stable, backend and frontend testing became easier because fewer unknowns remained. The workflow starts from YOLO-format food datasets and moves through class normalization, split generation, train-only augmentation, detector integration, nutrition import, class-to-food mapping, and finally API and interface development. Figure 3.1 summarizes the process.

This order was followed because early mistakes move forward into later stages. If one dish remains under two class names, the detector learns them as different outputs. If a detector class is linked to a wrong nutrition row, the frontend may still show a neat but incorrect result. For that reason, data checking and mapping verification received as much attention as model training.

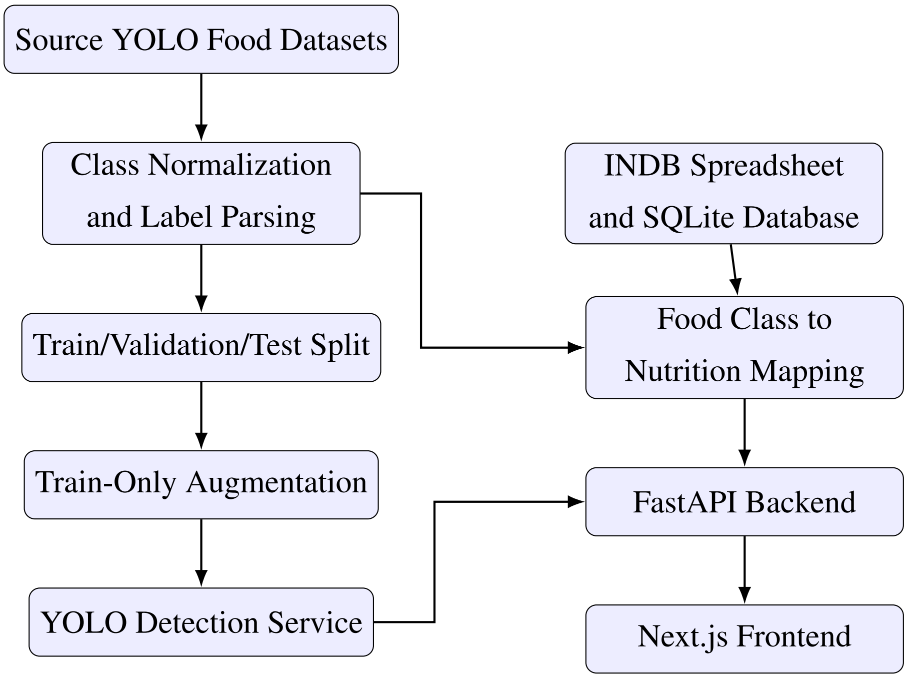


Figure 3.1: Methodological workflow for dataset, model, nutrition, backend, and frontend integration

3.2 YOLO Detection Approach

YOLO, short for You Only Look Once, was selected because it performs object detection in a single prediction pass. For a food image, the model predicts bounding boxes, class probabilities, and confidence scores together. This fits the web application because the backend has to return results quickly enough for the user to inspect the uploaded meal without a long delay.

The project uses the YOLO label format and inference style followed by recent Indian food detection systems such as IndianFoodNet and Diet Engine [1, 12]. During inference, the model returns candidate detections for visible food items. The backend applies confidence filtering and non-maximum suppression so that repeated boxes are removed before nutrition lookup. Each final detection contains a class name, confidence value, and normalized bounding box.

Speed was not the only reason for using YOLO. Indian meal photographs often contain more than one dish, and a whole-image classifier would usually miss side items. YOLO allows the application to detect multiple foods in the same image and map each detected class separately to the nutrition database. It also gives a useful debugging path: the image region, detected class, database row, and final macro values can be checked one by one.

3.3 Source Dataset Collection

Five YOLO-format datasets were used as source datasets. The local exports include Roboflow Universe metadata, dataset versions, source URLs, and CC BY 4.0 license notes [19–23]. Together, they contained 40,797 source images across training, validation, and test folders before project-specific merging. Combining them improved food coverage, but it also introduced naming and annotation issues. Some labels used CamelCase, some used spaces, some used regional words, and some referred to the same dish with different spellings. These differences were resolved before training so that the final model would not learn duplicate or confusing outputs. Table 3.1 summarizes the source data.

Table 3.1: Source datasets used for merged Indian food dataset

#	Source dataset	Classes	Train	Valid	Test
1	Indian food detection v1 [19]	17	7547	1053	453
2	Indian food v6 [20]	29	8088	213	105
3	Indian food v2 seven-class subset [21]	7	1698	159	92
4	IndianFoodNet YOLO export [22]	30	11387	1073	576
5	South Indian food detection v19 [23]	31	6478	1294	581

The raw class count was higher than the final class count because several labels referred to the same food. For example, `satham`, `rice`, and `WhiteRice` were normalized into `rice`. This was not just a naming cleanup. Object detectors learn one output channel per class, so equivalent labels left separate would train the model to treat the same food as different objects. The same normalized names also made the later nutrition mapping step more manageable.

3.4 Canonical Class Normalization

The canonical label set contains 72 food classes. Normalization converts raw source labels into snake-case names, resolves spelling differences, and merges regional variants. Some cases still required manual checking. A script can clean punctuation and casing, but it cannot always decide whether two regional names refer to the same dish. Table 3.2 shows examples.

Table 3.2: Examples of raw-to-canonical label normalization

Raw variants	Canonical class
`AlooGobi`, `aloo gobhi`, `aloo_gobi`	`aloo_gobi`
`CoconutChutney`, `thengai chutney`, `coconut chutney`	`coconut_chutney`
`Idli`, `idly`	`idli`
`WhiteRice`, `rice`, `satham`	`rice`
`lemon satham`	`lemon_rice`
`medu vadai`, `medu vada`	`medu_vada`

3.5 YOLO Label Representation

Each YOLO label file contains one row per annotated object:

$$y = (c, x, y, w, h) \tag{3.1}$$

where c is the class identifier, x and y are normalized center coordinates, and w and h are normalized width and height. The normalized coordinate system makes labels independent of image resolution. During parsing, each raw class identifier is first resolved to its source dataset class name and then converted to the canonical class name. This two-step conversion was necessary because class identifiers are local to each dataset and cannot be compared directly across sources.

3.6 Dataset Splitting and Augmentation

The merged dataset uses a 70/15/15 target ratio for training, validation, and testing. Multi-label images are represented through class-presence vectors. A stratified split is attempted first so that classes remain reasonably distributed across splits. When rare-label constraints make this infeasible, deterministic random splitting is used. The split stage was kept deterministic so that the same dataset can be regenerated and checked again.

Augmentation is applied only to the training split. This prevents transformed training samples from leaking into validation or test evaluation. The augmentation strategy uses horizontal flipping and mild brightness and contrast changes. The purpose was limited: increase coverage for underrepresented classes without creating food images that no longer look realistic. Stronger transformations were avoided because colour and texture are important cues for visually similar Indian dishes.

Algorithm 1 Dataset merge and export procedure

```
Initialize empty sample list S
for each source dataset D do
    Load source class names from data.yaml
    for each image-label pair in D do
        Parse YOLO labels
        Convert raw class names to canonical classes
        Add sample to S
    end for
end for
Build multi-label matrix for all samples in S
Split into train, validation, and test sets
Export images and labels using canonical class identifiers
Apply augmentation only to training samples below target count
Write final data.yaml, build log, and per-class count CSV
```

3.7 Nutrition Database Methodology

The nutrition database is built from the INDB spreadsheet. The required columns are `food_name`, `energy_kcal`, `protein_g`, `carb_g`, and `fat_g`. These columns are renamed to the runtime schema: `name`, `calories`, `protein_g`, `carbs_g`, and `fat_g`. A normalized food-name column is added for lookup and matching. Source metadata is also stored so that supplemental values can be separated from INDB rows.

The database contains two main tables. The first table, `indb_foods`, stores food records, macronutrients, and source information. The second table, `yolo_mappings`, stores precomputed mappings from YOLO classes to database food names. This separation keeps lookup fast and makes mapping decisions easier to inspect. It also allowed the mapping table to be regenerated during development without changing the original nutrition records.

3.8 Nutrition Mapping Methodology

Food-to-nutrition mapping is performed conservatively. Manual semantic mappings are tried first by checking database food names and choosing the closest nutritionally meaningful record. Verified supplemental rows are used only when the available INDB sheet has no safe entry for a dataset class. Fuzzy matching is kept as the final fallback and is used only when the score is high enough. This caution is necessary because two strings can look similar while the foods are nutritionally different, especially among sweets, curries, chutneys, and fried snacks.

This mapping step took more time than expected. Several detector labels were easy for a person to understand but did not match the database wording exactly. A regional or shortened class name could fail exact lookup, while fuzzy matching could select the wrong food because the words looked close. Since the final application displays numerical macro values, mapping was treated as a validation problem rather than a convenience feature.

For a detected class c, the mapping function can be represented as:

$$M_c = \begin{cases} \text{manual}(c), & \text{if a verified manual mapping exists} \\ \text{supplemental}(c), & \text{if a verified supplemental row exists} \\ \text{fuzzy}(c), & \text{if score}(c) \geq 0.78 \\ \emptyset, & \text{otherwise} \end{cases} \tag{3.2}$$

The current mapping covers all 72 dataset classes. The classes `bhakarwadi`, `ghevar`, `jalebi`, `khandvi`, and `nandu_masala` were absent from the available INDB sheet, so they are handled through verified supplemental nutrition rows with source metadata rather than weak fuzzy substitutes.

3.9 Macro Calculation Methodology

The macro engine converts a daily calorie target into grams of carbohydrates, protein, and fat. Each goal starts with a base macro split, and health conditions modify this split before normalization. The calculation was kept rule-based and explicit so it could be checked manually during testing. If T is the target calorie value and p_c, p_p, and p_f are the final percentages for carbohydrates, protein, and fat, then:

$$G_c = \frac{T \times p_c}{4 \times 100} \tag{3.3}$$

$$G_p = \frac{T \times p_p}{4 \times 100} \tag{3.4}$$

$$G_f = \frac{T \times p_f}{9 \times 100} \tag{3.5}$$

where G_c, G_p, and G_f are grams of carbohydrates, protein, and fat respectively. The values 4, 4, and 9 represent calories per gram for carbohydrates, protein, and fat. The generated values are later shown in the frontend as target macros against which the detected meal can be compared.

3.10 Validation Methodology

Validation is performed at multiple levels because one failing layer can make the final output misleading. Dataset validation checks class names, split counts, and visual class samples. Database validation checks row counts and schema. Mapping validation checks class-to-food mappings, coverage, and source provenance. Service validation checks whether the backend can initialize the nutrition service and return expected lookup results. Frontend validation checks whether the user workflow can display detections, nutrition labels, and recommendations. This layered approach made debugging manageable because problems could be isolated to the dataset, database, service, or interface instead of being treated as one large application error.

Chapter 4

SYSTEM DESIGN AND IMPLEMENTATION

4.1 Design Goals

The design was guided by five practical goals: usability, modularity, traceable data flow, local deployability, and extensibility. Usability means that a user should be able to upload a food image and understand the result without knowing how the model works internally. Modularity keeps dataset building, model inference, nutrition lookup, recommendation generation, and interface rendering separated enough for independent debugging. Traceable data flow keeps detected labels, confidence scores, bounding boxes, and nutrition sources visible instead of hiding them inside backend logic. Local deployability keeps the prototype easy to run on a developer machine, while extensibility leaves room for additional food classes, improved models, and better nutrition sources later.

These goals came from issues faced during development. Early testing showed that a food-recognition result is hard to trust when the label, confidence score, and nutrition source are not visible together. The design therefore avoids treating the detector as an unexplained black box. Each layer returns structured information that can be checked during debugging and shown clearly in the frontend.

4.2 High-Level Architecture

The project follows a layered architecture because each part has a different responsibility. The data layer contains the source datasets, merged YOLO dataset, INDB spreadsheet, and SQLite nutrition database. The AI layer contains the YOLO model and inference service. The backend layer exposes APIs and coordinates services. The frontend layer handles the user workflow. Figure 4.1 shows this architecture.

This layering reduced debugging time. When a wrong macro value appeared, the mapping table could be checked without retraining the detector. When the frontend display looked incorrect, the backend JSON response could be inspected separately. This separation made the project easier to manage as a full-stack application instead of one large script.

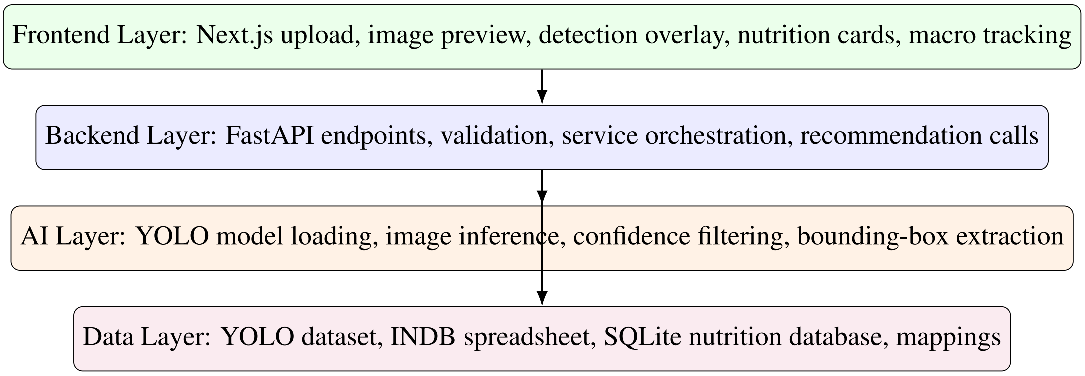


Figure 4.1: Layered architecture of the food recognition and nutrition system

4.3 Runtime Data Flow

The runtime flow begins when the user uploads an image. The frontend sends the raw image to the backend. The backend validates the upload, passes it to the vision service, receives detected objects, and looks up nutrition for each detected label. The response includes image dimensions, bounding boxes, labels, confidence values, macro values, and nutrition source metadata. The frontend then renders boxes over the image and displays nutrition cards so the user can compare the visual prediction with the nutrition values.

The response format was kept detailed deliberately. During testing, it was not enough to know that the application detected a food item. It was also necessary to check where the box was drawn, how confident the model was, which database row was selected, and whether the macro values came from INDB or a supplemental source.

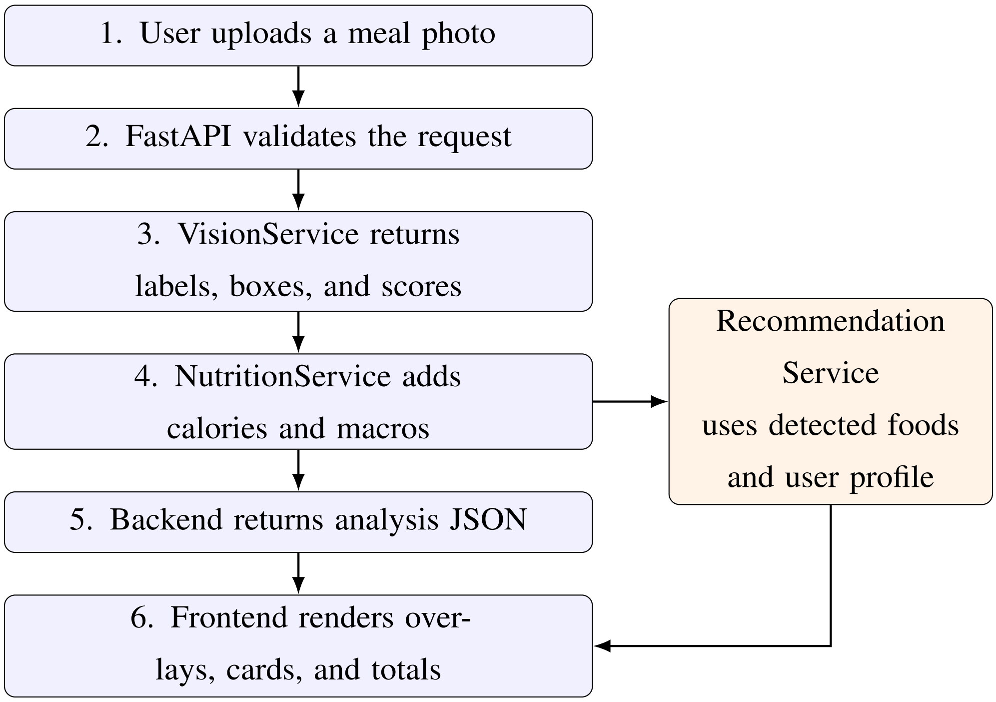


Figure 4.2: Runtime request-response flow

4.4 Backend Design

The backend is implemented with FastAPI, an ASGI-compatible Python framework suitable for typed request models, upload handling, and asynchronous service execution [24]. The backend contains the following core services:

- **VisionService**: loads the YOLO model and performs inference on uploaded images.
- **NutritionService**: connects to SQLite, caches food names, loads `yolo_mappings`, and returns macro values.
- **RecommendationService**: computes goal and health-condition context and generates dietary suggestions.

The backend separates the endpoint layer from service logic. Nutrition lookup can be tested without running image inference, and macro calculation can be tested without uploading files. This separation was useful while debugging the nutrition layer because lookup errors could be reproduced with a single food name instead of repeating the complete image-upload workflow.

4.5 Frontend Design

The frontend is built with Next.js 14 and React [25]. It follows one main workflow: upload an image, analyze it, inspect detections, adjust serving estimates, review macro impact, and request recommendations. Frontend state is organized around the image analysis result, selected goal, target calories, selected health condition, and macro totals.

The interface opens directly to the working tool rather than a marketing-style page. The first screen contains upload controls, user nutrition context, and result panels. This fits the purpose of the application because it is meant for checking a meal, not for browsing product information. During testing, the most useful interface behaviour was seeing the uploaded image and nutrition cards together, because it quickly showed whether the visual prediction and database output matched.

4.6 Database Design

The SQLite database contains a normalized nutrition table and a precomputed mapping table. Table 4.1 summarizes the database design.

Table 4.1: Runtime nutrition database tables

Table	Key columns	Purpose
`indb_foods`	id, food name, normalized name, calories, protein, carbohydrates, fat, source	Stores INDB and supplemental nutrition records.
`yolo_mappings`	YOLO class, matched food name, match score	Stores precomputed mappings from detector labels to database food rows.

The database is intentionally simple. A heavier database server would add setup work without improving the core prototype. Since the nutrition data is mostly read-only at runtime, SQLite is sufficient and keeps the project easy to run locally. It also makes the generated database file easy to inspect after rebuilding the mapping table.

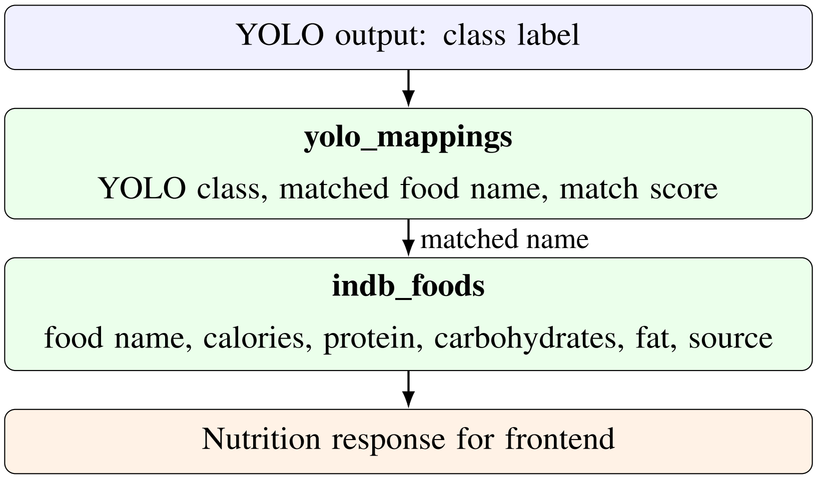


Figure 4.3: Entity relationship used for class-to-nutrition lookup

4.7 API Design

The API exposes endpoints for health checks, image analysis, nutrition testing, nutrition validation, macro calculation, goal metadata, health-condition metadata, and recommendations. The testing and validation endpoints were kept because they helped verify the system during development and can help future maintenance. Table 4.2 lists the implemented endpoints.

Table 4.2: Implemented backend endpoints

Endpoint	Method	Purpose
`/`	GET	Root API description and endpoint list.
`/api/health`	GET	Returns service health and model-loaded status.
`/api/analyze-food`	POST	Accepts image upload, runs YOLO inference, and attaches nutrition data.
`/api/test-nutrition/<food_name>`	GET	Debug endpoint for a single food nutrition lookup.
`/api/validate-nutrition`	GET	Validates nutrition lookup coverage for model classes.
`/api/user/calculate-macros`	POST	Calculates goal-specific macro targets.
`/api/user/goals`	GET	Returns supported dietary goals.
`/api/user/health-conditions`	GET	Returns supported health condition options.
`/api/recommendations`	POST	Generates dietary recommendations from detected foods and user profile.

4.8 Security and Validation Design

The system validates upload type, empty uploads, maximum file size, and service readiness. The backend enforces a maximum image upload size of 10 MB. Rate limiting is implemented for the food analysis endpoint to avoid repeated heavy inference calls. Secrets such as the Groq API key are loaded from environment variables and are not written in the report.

These checks are basic, but they matter in a working prototype. Without them, a non-image file, an empty upload, or an unloaded model can produce unclear errors in the frontend. Returning structured failures makes the application easier to test and prevents the interface from showing incomplete nutrition results.

4.9 Design Trade-Offs

The design uses per-100-gram nutrition values instead of automatic portion estimation. This limits calorie accuracy, but it avoids pretending that one image is enough to recover food mass reliably. Monocular portion estimation would require plate scale, camera distance, depth, and food-volume assumptions. The frontend therefore uses serving multipliers so the user can adjust values when the visible quantity is clearly smaller or larger than the baseline.

The design also keeps explicit nutrition mappings in the database build process. This is less automatic than fuzzy matching, but it is safer for a nutrition-related application. A wrong nutrition value can be more misleading than a missing one. For that reason, classes without safe INDB rows are handled only through verified supplemental entries with source metadata. The prototype becomes slightly more manual to maintain, but the output is easier to control.

4.10 Development Environment

The project was implemented as a full-stack application rather than only a training notebook. This made the final work closer to a usable prototype, but it also meant that the detector, nutrition database, API response format, and frontend display had to remain consistent with each other. A small class-name change in the dataset, for example, also required checking the nutrition mapping and display name. The backend uses Python, FastAPI, PyTorch, Ultralytics YOLO, Pillow, NumPy, Pandas, OpenPyXL, SQLite, and AioSQLite. The frontend uses Next.js, React, TypeScript, Tailwind CSS, and Lucide icons. The repository is organized into `backend/`, `frontend/`, `data/`, `models/`, `scripts/`, and `notebooks/` so that model assets, data processing, API code, and interface code remain separated.

4.11 Dataset Build Script

The dataset build script is located at `scripts/build_master_dataset.py`. It handles the work that would be difficult to repeat manually: source dataset scanning, class-name loading, YOLO label parsing, canonical class conversion, multi-label split creation, image export, training augmentation, and artifact writing. It writes three key outputs: `data.yaml`, `build_log.txt`, and `split_class_counts_before_after.csv`.

This script became one of the most important implementation pieces because the source datasets did not follow one naming convention. Some files had class names embedded in filenames, some labels had to be read through dataset-specific `data.yaml` files, and several raw names referred to the same dish. Automating the merge reduced manual copying mistakes and made it possible to rebuild the dataset after correcting normalization rules.

The final dataset contains 72 canonical classes. The generated balanced dataset contains 36,852 training image-label pairs, 5,922 validation pairs, and 5,919 test pairs. The total exported image-label pairs are 48,693. The larger secondary export retained in `data/master_dataset/` contains more aggressive train augmentation and is documented as an archived artifact.

4.12 Final YOLO Training Notebook

The final model training workflow is recorded in the final training notebook under `notebooks/`. The notebook loads the generated 72-class `data.yaml` and records the YOLO11s training workflow used for the final 100-epoch result. For the report, the detector metric values are taken from the consolidated artifacts in `merge_results/`. The configuration uses 640-pixel images, batch size 16, automatic mixed precision, checkpoint saving every 10 epochs, and plot generation enabled.

The deployed model was not selected simply from the last epoch. The best exported weight was used because validation mAP reached its strongest value before the final logged epoch. The `merge_results/` folder was kept with the exported model so that the reported precision, recall, mAP values, and plotted curves could be checked against the saved artifacts.

The run used PyTorch 2.10.0 with CUDA 12.8 on a Tesla T4 GPU. The exported artifacts include `best.pt`, `last.pt`, checkpoint weights, training curves, precision-recall curves, confusion matrices, prediction batches, and the consolidated `merge_results/merged_results_1_to_100.csv`. The best model file was also exported as `yolo11s_indian_food_best.pt`, which is the detector artifact used for final evaluation.

4.13 Visual Class Verification Implementation

To verify that class names correspond to actual images, a visual verification script was created at `scripts/verify_food_classes_with_images.py`. The script groups exported images by the canonical class suffix in the filename, samples four images per class, parses corresponding labels for object crops, and writes contact sheets. This step was useful because file counts alone cannot show whether a class name visually matches the sampled images. The original eight sheets were split into sixteen half-sheets so that labels and samples remain readable in print. The full half-sheet set and the detailed annotation-issue table are placed in Appendix A to avoid repeating the same visual material in the implementation chapter.

The visual review confirmed that all 72 classes have actual image samples and that the class-name list is aligned. It also exposed a few dataset issues, such as a crop that did not match the expected food or a box placed on the wrong side item. These were not class-order failures, but they still matter because object detectors learn directly from the boxes provided during training.

4.14 Nutrition Database Build Implementation

The nutrition database build script is located at `backend/scripts/merge_db.py`. It reads the INDB spreadsheet, verifies required columns, normalizes food names, removes duplicate normalized names, converts nutrition columns to numeric values, and rebuilds the local SQLite nutrition database. The script then loads the current dataset classes and appends legacy model labels so that older deployed labels still resolve correctly.

Manual mappings are applied before fuzzy matching. This prevents incorrect string-based matches. For example, `aloo_gobi` maps to `Potato cauliflower (Aloo gobhi)`, `palak_paneer` maps to `Spinach paneer (Palak paneer)`, and `rice` maps to `Boiled rice (Uble chawal)`. The script also adds source-backed supplemental rows for detector classes absent from the available INDB sheet. This required more checking, but it avoided several wrong matches found in early tests.

4.15 Vision Service Implementation

The vision service loads YOLO model weights and processes uploaded images. It converts input bytes into an image representation, performs inference, applies class-level confidence rules, extracts bounding boxes, and returns structured detection dictionaries. Each detection contains the food label, confidence score, bounding box, and image dimensions.

The model is warmed up during backend startup with a dummy image. This reduces first-request latency, which otherwise can make the first upload feel unusually slow. The backend executes inference in a thread pool so that the asynchronous API loop is not blocked by model computation. Keeping the output as dictionaries also made it easier to attach nutrition data without changing the model inference code.

4.16 Nutrition Service Implementation

The nutrition service initializes by connecting to `data/nutrition.db`. It caches all food names and normalized names and loads the `yolo_mappings` table. Runtime lookup follows this order:

1. Check precomputed YOLO mapping.
2. Try exact normalized-name match.
3. Try partial normalized match.
4. Generate alias-based variations.
5. Use fuzzy matching as the last fallback.

The precomputed mapping step is the most important because it avoids incorrect fuzzy matches. The returned nutrition object includes display name, macro values, unit, source, and food-found status. During testing, this status field helped the frontend distinguish between a detected food with nutrition data and a detected food whose nutrition record was missing or uncertain.

4.17 Recommendation and Macro Implementation

The macro calculation function accepts a target calorie value, a selected goal, and a health condition. It applies base goal splits and health-condition modifiers, normalizes the result, and converts calories into grams. The recommendation service uses detected foods and user context to generate three practical suggestions. The prompt includes detected foods, calories, goal, health profile, and health-specific dietary guidance. The calculation was kept separate from recommendation text so that macro targets can be tested even when the external recommendation service is not configured.

4.18 Frontend Implementation

The frontend is implemented in `frontend/app/page.tsx` and reusable components. `ImageUpload` manages drag-and-drop upload and file preview. `ResultsPanel` displays detection cards, nutrition values, serving controls, macro totals, and recommendations. `NutritionLabel` displays calories, protein, carbohydrates, and fat. `ImageOverlay` draws bounding boxes over the uploaded image.

The serving controls were included because the detector can identify a food item but cannot know its exact mass from a single image. This keeps the interface honest: the application starts with a calculated baseline, and the user can adjust the assumed serving when the visible quantity is clearly larger or smaller.

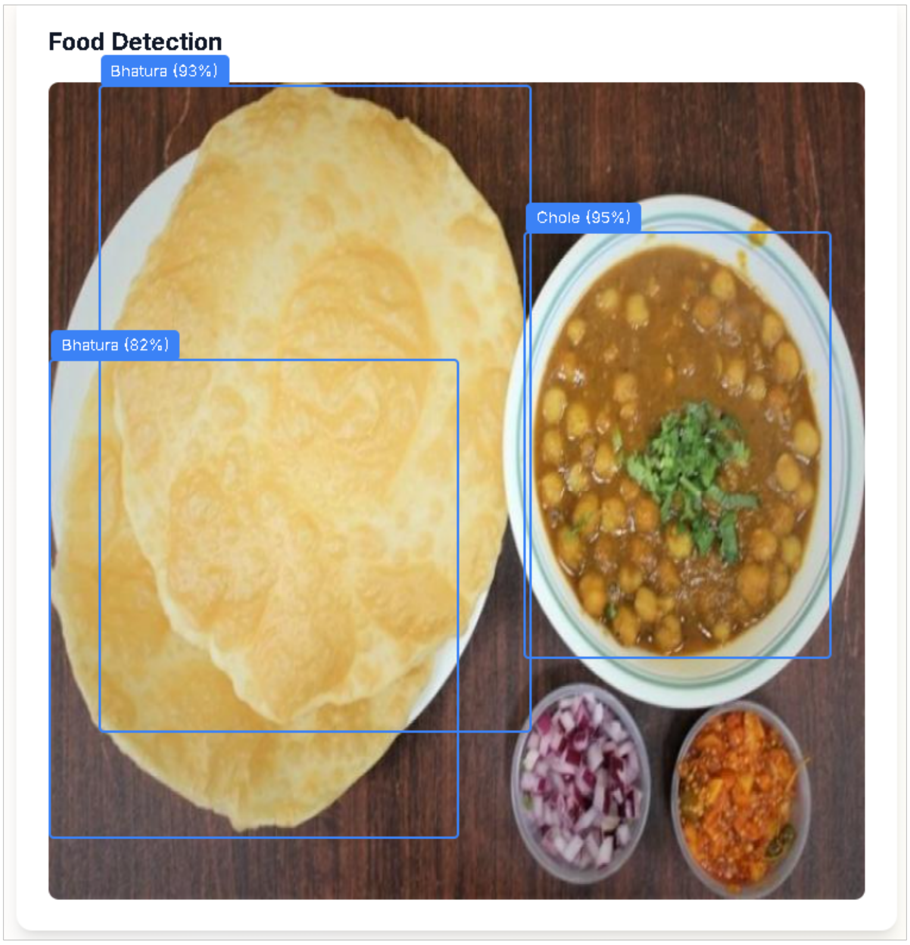


Figure 4.4: Frontend detection overlay showing Bhatura and Chole detections

4.19 Code Organization

Table 4.3 summarizes the important implementation files.

Table 4.3: Important implementation files

File or component	Purpose
`backend/main.py`	FastAPI application, endpoints, startup sequence, request validation.
`vision_service.py`	YOLO model loading, inference, and detection formatting.
`nutrition_service.py`	SQLite lookup, mapping cache, and nutrition response creation.
`recommendation service`	Goal-aware and health-condition-aware recommendations.
`merge_db.py`	INDB import, supplemental nutrition rows, and YOLO-to-database mapping creation.
`food_normalizer.py`	Food-name normalization and variation generation.
`dataset build script`	Merged dataset construction and augmentation.
Class verification script	Contact-sheet generation for visual class verification.
`page.tsx` and UI components	Main frontend workflow, detection cards, serving controls, and macro display.

4.20 Implementation Challenges

The main challenges were label inconsistency, nutrition mapping ambiguity, and multi-component integration. Label inconsistency was handled through canonical class naming. Nutrition ambiguity was handled through explicit mappings and conservative fallback. Integration complexity was reduced by separating services and keeping JSON response shapes structured.

In practice, the mapping work needed more attention than expected because a visually correct food label can still produce the wrong macro values if it is linked to the wrong database row. Another challenge was keeping training artifacts, deployed model names, dataset classes, and frontend labels synchronized. Small mismatches did not always cause immediate failures, but they produced confusing results during testing. For this reason, validation scripts and debug endpoints were kept in the project instead of being removed after the first successful run.

Chapter 5

EXPERIMENTAL ANALYSIS AND DISCUSSION

5.1 Experimental Scope

The experiments in this project are based on repository files, generated reports, and exported training artifacts. The evaluation covers dataset construction, YOLO11s detector training, validation and test metrics, class-name verification, nutrition database integrity, mapping coverage, backend service behavior, frontend workflow, and qualitative detection output. For detector reporting, the `merge_results/` folder is used as the metric source. It contains the consolidated 100-epoch CSV, the training-curve image, and separate validation and test image-evaluation folders with PR, F1, precision, recall, confusion-matrix, and prediction-batch artifacts for the latest 72-class model.

Validation was not limited to checking whether the model produced a score. Several parts of the application had to work together. A detection result was useful only if the class existed in the final `data.yaml`, the nutrition service could resolve the class name, and the frontend could display the returned macro fields correctly.

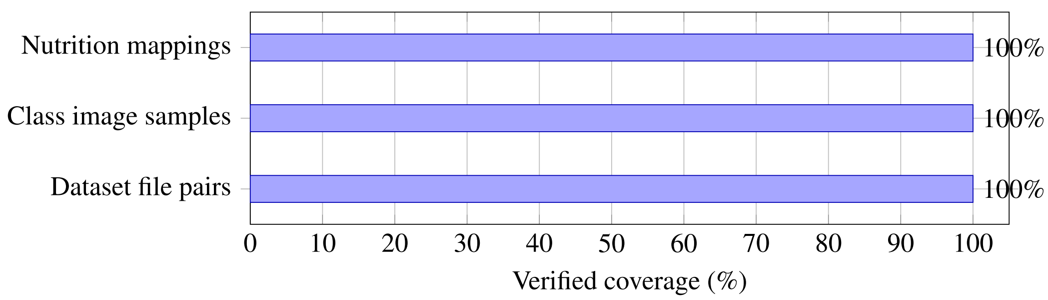


Figure 5.1: Verified system-level coverage used for validation

5.2 Dataset Validation

The first validation task checked whether the merged dataset contained the expected split folders and a complete 72-class `data.yaml`. This step was necessary because a missing label file or class-count mismatch would make later training results unreliable. The generated dataset summary is shown in Table 5.1.

Table 5.1: Generated dataset summary

Property	Value
Canonical classes	72
Train images	36,852
Train labels	36,852
Validation images	5,922
Validation labels	5,922
Test images	5,919
Test labels	5,919
Total exported image-label pairs	48,693
Split target ratio	70/15/15

The split counts show that every exported image has a corresponding label file. Validation and test augmentation are disabled so that held-out samples remain real images rather than transformed versions of training data. As a result, the validation and test metrics are not based on augmented copies.

5.3 YOLO Training and Detector Evaluation

The final detector experiment used YOLO11s on the 72-class dataset and reports the final 100-epoch training and evaluation outcome. Table 5.2 summarizes the training setup used for the run, while the reported metric values are taken from the consolidated artifacts stored in `merge_results/`.

Table 5.2: Final YOLO11s training setup

Property	Value
Model	YOLO11s object detector
Classes	72
Image size	640 pixels
Batch size	16
Epochs reported	100
Training-curve source	Consolidated curve artifact from `merge_results/`
Hardware	Tesla T4 GPU, 15.64 GB VRAM
Framework	Ultralytics 8.4.60, PyTorch 2.10.0, CUDA 12.8
Best exported weight	`yolo11s_indian_food_best.pt`

The final training log shows that training and validation losses reduced with normal epoch-to-epoch variation. The training metrics were checked from `merge_results/merged_results_1_to_100.csv`. The highest validation mAP@50 in this CSV file was 0.83379 at epoch 78, and the highest mAP@50:95 was 0.69226 at epoch 97. The final epoch recorded precision 0.85280, recall 0.80420, mAP@50 0.82806, and mAP@50:95 0.69164. Since the best validation value occurred before the final epoch, the exported best checkpoint was used for evaluation. The plotted training curves are shown separately using the consolidated curve artifact in Figure 5.2.

Table 5.3: Training metric validation from the merged 100-epoch CSV

Checked item	Epoch	Verified values
Best mAP@50 row	78	Precision 0.84685, recall 0.80852, mAP@50 0.83379, mAP@50:95 0.68440.
Best mAP@50:95 row	97	Precision 0.84696, recall 0.80721, mAP@50 0.82999, mAP@50:95 0.69226.
Final logged row	100	Precision 0.85280, recall 0.80420, mAP@50 0.82806, mAP@50:95 0.69164.

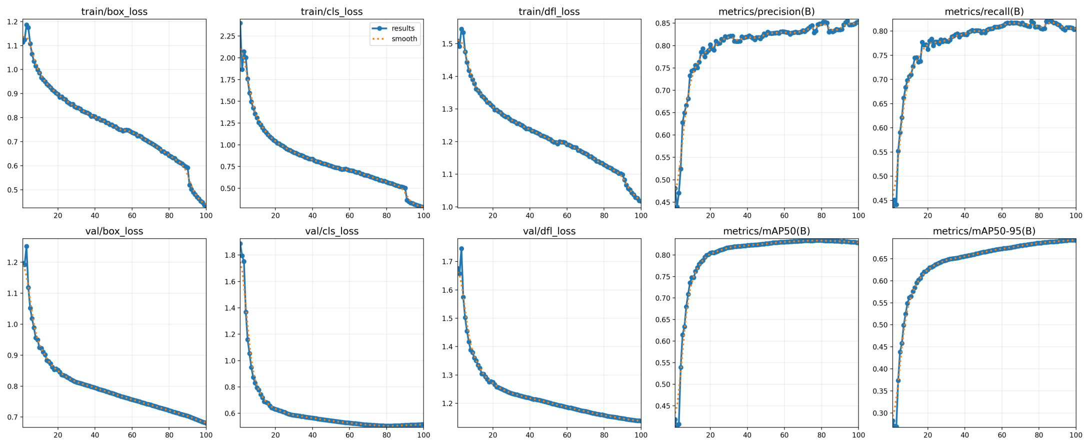


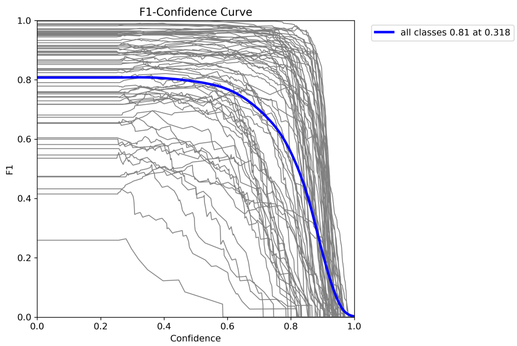
Figure 5.2: YOLO11s training result curves for the complete 100-epoch run

The best exported model was evaluated on both validation and held-out test images. The split-level values in Table 5.4 are read only from the image-evaluation artifacts inside `merge_results/valid_validation_metrics/` and `merge_results/test_validation_metrics/`. These folders contain the plotted metric curves and prediction outputs. They do not contain a separate numeric mAP@50:95 summary file, so mAP@50:95 is reported from the 100-epoch training CSV only and is not assigned to the split-level image-evaluation rows.

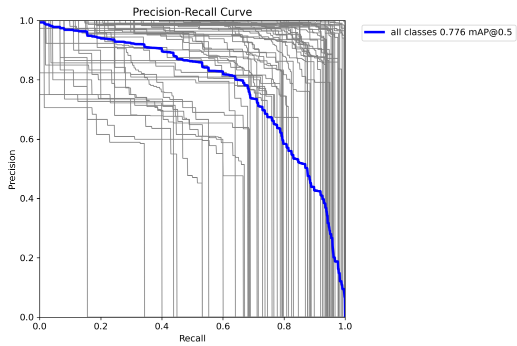
Table 5.4: YOLO11s split-level image-evaluation metrics from merge results

Split	Artifact	mAP50	Curve values read from legends
Validation	Validation curves	0.776	Best F1 0.81 at confidence 0.318; maximum recall 0.82 at confidence 0.000; maximum precision 1.00 at confidence 0.976.
Held-out test	Test curves	0.765	Best F1 0.80 at confidence 0.343; maximum recall 0.81 at confidence 0.000; maximum precision 1.00 at confidence 0.971.

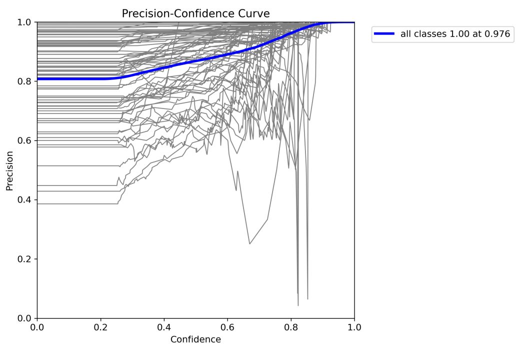
The exported run also produced confidence and precision-recall curves for bounding-box evaluation. These plots were useful during interpretation because they show how the model behaves across confidence thresholds instead of relying only on one aggregate value. Figure 5.3 shows the validation image-evaluation curves, and Figure 5.4 shows the held-out test image-evaluation curves.



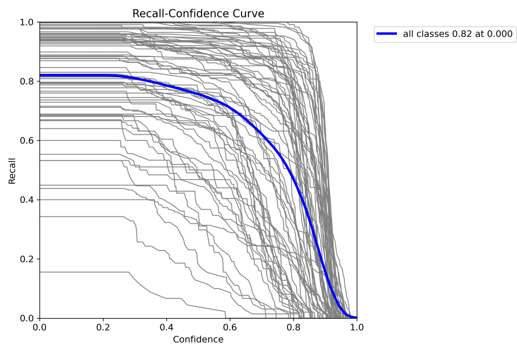
(a) F1-confidence



(b) Precision-recall

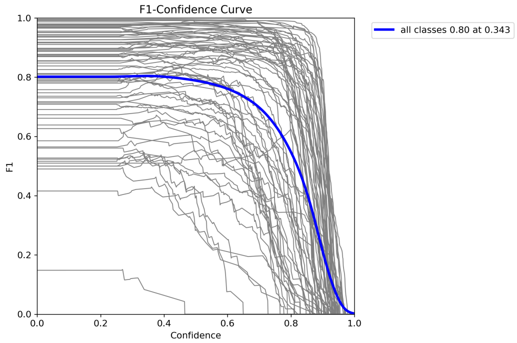


(c) Precision-confidence

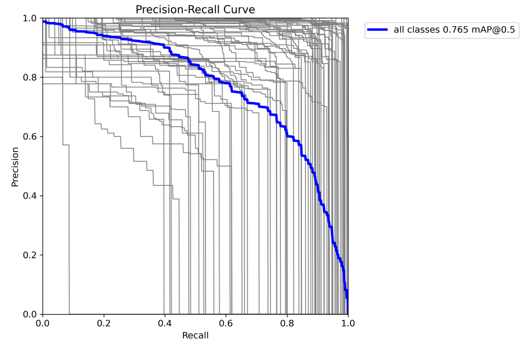


(d) Recall-confidence

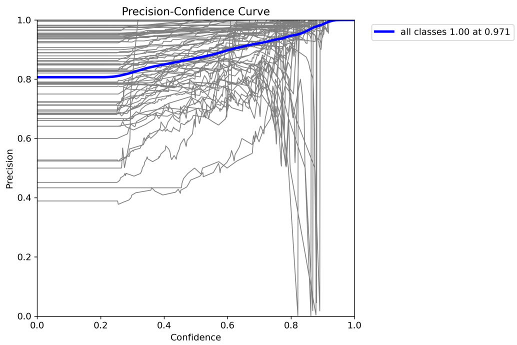
Figure 5.3: YOLO11s validation image-evaluation curves



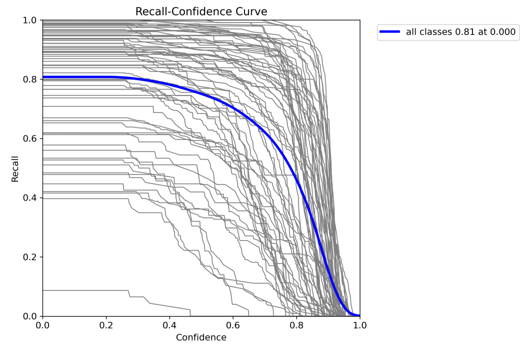
(a) F1-confidence



(b) Precision-recall



(c) Precision-confidence



(d) Recall-confidence

Figure 5.4: YOLO11s held-out test image-evaluation curves

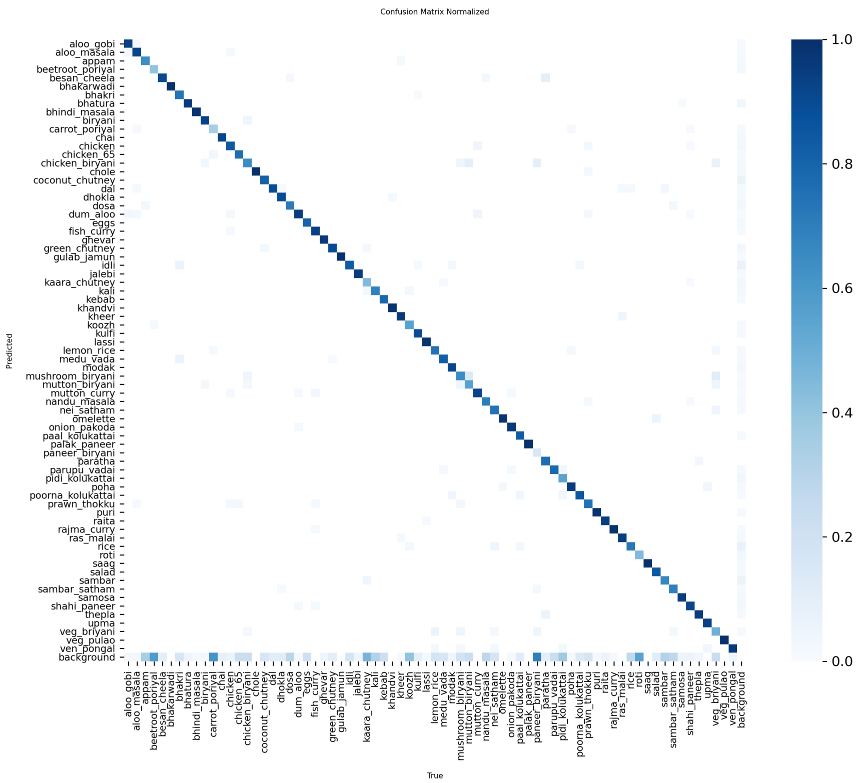


Figure 5.5: Normalized confusion matrix for the YOLO11s validation image evaluation

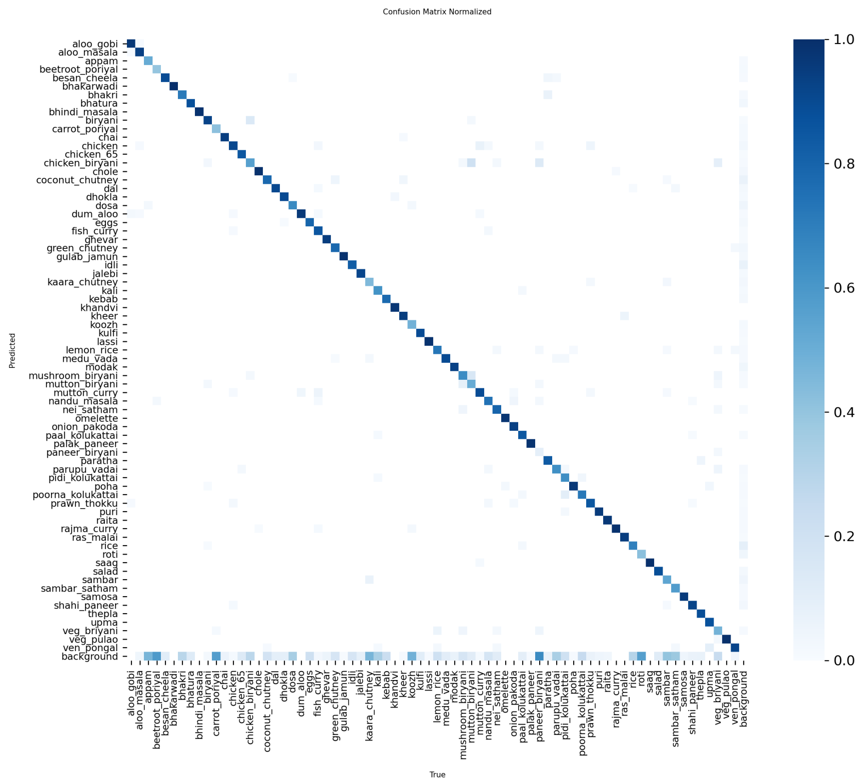


Figure 5.6: Normalized confusion matrix for the YOLO11s held-out test evaluation

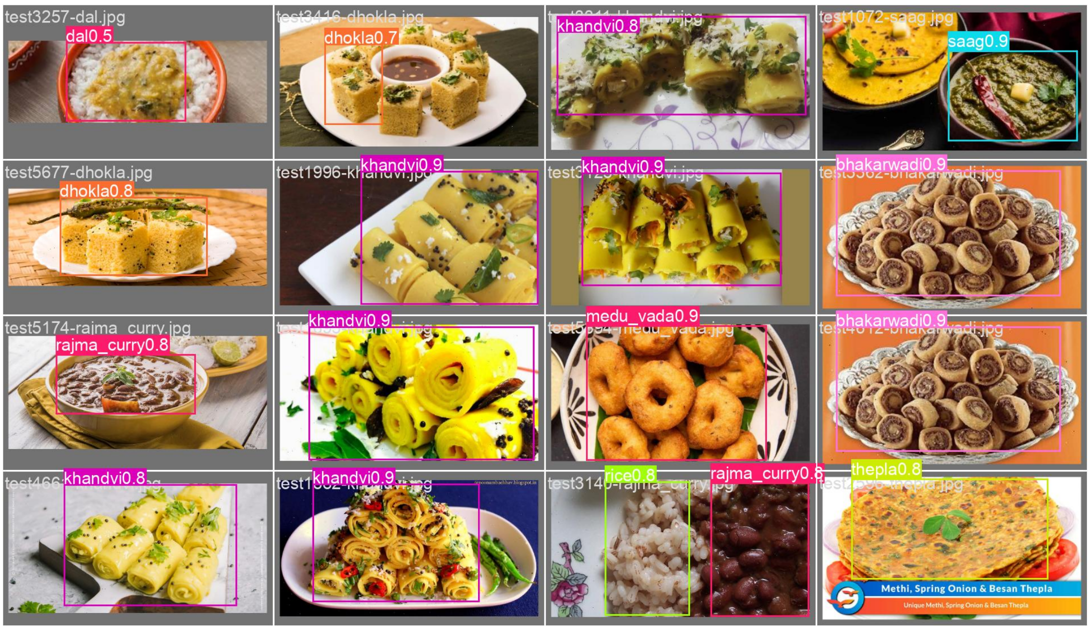


Figure 5.7: Sample YOLO11s predictions from the held-out test split

The split-level image-evaluation metrics show that the held-out test mAP@50 is close to the validation mAP@50, with a drop from 0.776 to 0.765. The best all-class F1 score also changes only slightly, from 0.81 on validation to 0.80 on the held-out test split.

The normalized confusion matrices still show uneven class behaviour, which is expected in a 72-class food detector where several dishes look similar and some classes have fewer unique source images. During visual inspection, many mistakes were understandable when foods shared colour, texture, or serving style, such as rice-based dishes, paneer gravies, and chutney-side combinations.

5.4 Class Verification Experiment

The class verification experiment generated contact sheets with four images per class. The purpose was to catch practical mistakes that are easy to miss in code, such as class-order shifts, naming mismatches, or labels that do not match the visible food. The script found image samples for all 72 classes. Visual inspection found no class-list order problem. The issues observed were sample-level annotation issues rather than class-list errors.

This experiment was useful because it gave a quick human-readable check of the dataset. CSV counts alone would not show whether a `jalebi` sample actually looked like jalebi or whether a chutney box was placed on the correct food region. The contact sheets made these problems visible without opening hundreds of files manually.

Table 5.5: Visual class verification result

Check	Result
Classes checked	72
Classes with image samples	72
Classes without image samples	0
Contact sheets generated	8
Displayed samples per class	4
Class-order problem found	No
Sample-level issues found	3

5.5 Nutrition Database Validation

Nutrition database validation checked schema and row counts. The database contains 1,010 records in `indb_foods` and 103 records in `yolo_mappings`. The food table is based on INDB and five source-backed supplemental rows for classes absent from INDB. The mapping count is larger than 72 because the table includes expanded dataset classes and legacy deployed-model labels for backward compatibility.

The row-count check was followed by lookup tests because a database can contain records and still return the wrong food for a detector label. This was especially important after the mapping logic was changed from mostly fuzzy matching to explicit class-to-food mappings.

Table 5.6: Nutrition database validation result

Table	Purpose	Count
`indb_foods`	INDB plus supplemental nutrition records	1,010
`yolo_mappings`	YOLO class to database mappings	103

5.6 Nutrition Mapping Coverage Experiment

The mapping coverage experiment checked how many of the 72 dataset classes have a safe nutrition mapping. The result is shown in Table 5.7. Coverage was measured on the final canonical class list rather than only on currently detected labels, because the model can return any of the 72 classes during use.

Table 5.7: Nutrition mapping coverage for 72-class dataset

Metric	Value
Total dataset classes	72
Mapped classes	72
Unmapped classes	0
Mapping coverage	100.00%

The classes `bhakarwadi`, `ghevar`, `jalebi`, `khandvi`, and `nandu_masala` were not present as safe INDB rows, so verified supplemental entries were added with source metadata. The supplemental values use a FatSecret product nutrition label for `bhakarwadi` and Clearcals recipe nutrition pages for the remaining dishes [26–30]. This keeps the full 72-class dataset usable while avoiding unsafe fuzzy substitutions. These rows are documented as supplemental values and are not treated as equal to primary INDB coverage in the discussion.

Table 5.8: Verified supplemental nutrition rows

Class	Database row	kcal	Protein	Carbs	Fat
`bhakarwadi`	`Bhakarwadi`	510.0	10.76	57.03	26.55
`ghevar`	`Ghevar`	351.2	3.5	39.6	19.9
`jalebi`	`Jalebi`	316.8	3.4	44.6	13.8
`khandvi`	`Khandvi`	232.7	9.3	21.7	12.1
`nandu_masala`	`Nandu Kari (Crab masala)`	128.1	7.0	7.0	8.0

5.7 Service Lookup Validation

Service-level lookup validation was performed for classes that previously suffered from incorrect fuzzy matches. These examples were selected because they represented real failures observed during development, not only random test cases. The corrected results are shown in Table 5.9.

Table 5.9: Corrected nutrition lookup examples

Input class	Returned INDB display name
`aloo_gobi`	`Potato cauliflower (Aloo gobhi)`
`palak_paneer`	`Spinach paneer (Palak paneer)`
`rice`	`Boiled rice (Uble chawal)`
`AlooGobi`	`Potato cauliflower (Aloo gobhi)`
`WhiteRice`	`Boiled rice (Uble chawal)`

This confirms that the revised database mapping handles both canonical dataset labels and legacy deployed-model labels. It also confirms that the nutrition service can resolve common casing and naming differences such as `AlooGobi` and `WhiteRice`.

5.8 Backend API Validation

Backend validation focused on service readiness and response structure. The analysis endpoint returns HTTP 503 if vision or nutrition services are unavailable. Empty images, non-image MIME types, and uploads exceeding 10 MB are rejected. The analysis response includes food label, confidence, bounding box, nutrition source, macro values, and food-not-found status. This structured response is important because the frontend depends on predictable fields for visualization and macro tracking. It also makes debugging easier when a detection is correct but the nutrition object is missing or incomplete.

5.9 Frontend Workflow Validation

The frontend workflow was validated qualitatively by uploading sample food images and checking that the interface displays bounding boxes, detected labels, confidence values, nutrition cards, serving controls, and recommendation text. Figure 4.4 in the implementation section shows a successful example with Bhatura and Chole detections. The interface also handles no-food responses and backend error messages.

This workflow test was important because some issues appear only after the model response reaches the browser. Box coordinates must align with the displayed image size, serving changes must update macro totals, and recommendation text must use the current user goal rather than an old state.

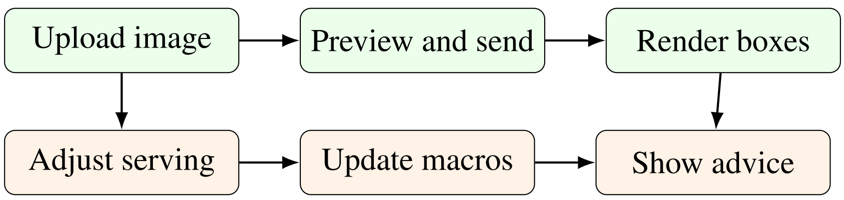


Figure 5.8: Frontend workflow checks used during qualitative validation

5.10 Macro Calculation Validation

Macro validation checked whether a target calorie value is converted correctly to grams. For example, for a 2,000 kcal weight-loss goal with a 40/35/25 split, the carbohydrate target is 200 g, protein target is 175 g, and fat target is approximately 56 g. Health profiles modify these base splits. The formula was discussed in Chapter 3. These calculations are simple enough to verify manually, which is useful because they directly affect the recommendation context shown to the user.

5.11 Experimental Findings

The experiments show that the dataset and application pipeline are internally consistent. In `merge_results/`, the consolidated training CSV records a best validation mAP@50 of 0.83379 and best validation mAP@50:95 of 0.69226, while the latest split-level image-evaluation curves report 0.776 mAP@50 on validation images and 0.765 mAP@50 on held-out test images. The same validation sequence also confirmed that all 72 expanded dataset classes have nutrition mappings and that the frontend can display detections, macro values, serving controls, and recommendation text. The next sections synthesize these observations and discuss their practical meaning in the complete system.

5.12 Integrated Result Synthesis

The previous sections report the individual counts, metrics, lookup tests, and workflow validations. Table 5.10 connects that evidence with its practical meaning in the complete system.

Table 5.10: Integrated result synthesis

Component	Verified evidence	Practical interpretation
Dataset	72 canonical classes and 48,693 exported image-label pairs in Table 5.1.	The model, nutrition database, backend response, and frontend labels share one stable class space.
Class verification	All 72 classes had visual samples, with no class-order problem in Table 5.5.	The dataset configuration is usable for training, although a few annotation-level issues remain.
Detector	Validation image-evaluation mAP@50 of 0.776 and held-out test mAP@50 of 0.765 in Table 5.4.	The detector generalizes reasonably across the project splits, with a small drop on the held-out test set.
Nutrition mapping	72 mapped classes and 100.00% coverage in Table 5.7.	Every detector class can be connected to a nutrition record, avoiding blank output for supported classes.
Lookup correction	Problem labels such as `aloo_gobi`, `palak_paneer`, and `rice` resolve to suitable database rows in Table 5.9.	Explicit mappings are safer than relying mainly on fuzzy string similarity.
Application workflow	Backend validation, frontend workflow checks, and Figure 4.4.	The project works as an end-to-end prototype rather than only as an isolated model experiment.
Macro and advice layer	Manual macro validation for a 2,000 kcal example and structured recommendation inputs.	Recommendation text is grounded in calculated macros and user context instead of unsupported nutrient guessing.

5.13 Dataset Coverage in the Final Label Space

The dataset result is not only a larger image count. The more important outcome is that the final label space covers common Indian meal components using normalized names that can be shared by the detector and nutrition service. Table 5.11 gives representative groups from the 72-class set.

Table 5.11: Representative categories in the 72-class dataset

Category	Example classes
Rice dishes	`rice`, `lemon_rice`, `sambar_satham`, `veg_briyani`, `veg_pulao`, `ven_pongal`
Flatbreads and fried breads	`roti`, `paratha`, `bhakri`, `bhatura`, `puri`, `thepla`
Curries and gravies	`chole`, `rajma_curry`, `fish_curry`, `mutton_curry`, `shahi_paneer`, `palak_paneer`
Snacks	`samosa`, `onion_pakoda`, `medu_vada`, `parupu_vadai`, `bhakarwadi`, `khandvi`
Sweets and desserts	`gulab_jamun`, `ras_malai`, `kheer`, `kulfi`, `modak`, `jalebi`
Chutneys and sides	`coconut_chutney`, `green_chutney`, `kaara_chutney`, `raita`, `salad`

The class groups also show why a single whole-image classifier would have been weak for this project. A normal meal image may contain rice, curry, chutney, and fried bread together. Treating the full image as one label would hide these side items, while object detection lets the backend map each visible item to a separate nutrition record.

5.14 Result Interpretation

The detector results are suitable for a final-year prototype, but they should be read together with the confusion matrix and prediction samples. The split-level image-evaluation curves in `merge_results/` show close validation and held-out test mAP@50 values, which suggests that the model is not fitting only to one split. At the same time, visually similar dishes remain difficult. This matters because a wrong detector label can move through the rest of the pipeline and produce a wrong nutrition lookup even when the backend and frontend are functioning correctly.

The nutrition mapping result is one of the stronger outcomes of the project. Earlier fuzzy matching tests produced misleading matches for some dish names. The final mapping strategy corrects these cases through explicit class-to-food mappings and uses source-backed supplemental rows only when the INDB sheet has no safe equivalent. Some mappings still remain approximate, such as `ven_pongal` to `Plain khitchdi` and `chicken_biryani` to `Chicken pulao`. These assumptions are kept visible through mapping confidence and source metadata rather than being hidden behind a confident interface.

The application result confirms that the pipeline is usable from upload to output. The frontend can display bounding boxes, confidence values, nutrition cards, serving controls, macro totals, and recommendation text. This is important because model accuracy alone does not prove that a user can understand or correct the result. The serving control is especially necessary because the detector identifies food type, not exact food mass.

5.15 Discussion

The results show that the project should be interpreted as a complete recognition-to-nutrition workflow. The detector identifies visible food regions, the nutrition layer decides what those labels mean in calories and macros, and the frontend makes the result visible enough for a user to inspect. If any one of these parts fails, the final output becomes less useful even when the other parts work correctly.

The strongest part of the system is traceability across this chain. A detected label can be followed to a bounding box, confidence score, mapped database row, nutrition source, macro values, and recommendation context. This was useful during development because errors could be isolated more easily. For example, a wrong macro value could be checked in the mapping table without retraining the detector, and a frontend display issue could be checked against the backend JSON response.

The project also benefits from using a domain-specific label space. Indian food names often differ by region, spelling, or common usage. Normalizing names such as `satham`, `medu_vadai`, and `thengai_chutney` made the dataset easier to train on and made nutrition lookup more controlled. The same normalization work supports the recommendation layer because the advice receives structured food and macro information rather than uncertain free-text meal descriptions.

5.16 Limitations and Practical Considerations

The main limitation is portion size. The system reports nutrition per 100 grams and allows serving adjustment, but it does not estimate exact food mass from one photograph. This was kept as an explicit boundary because reliable portion estimation would require extra information such as depth, a reference object, plate size, segmentation masks, or learned volume estimation. As a result, the calorie and macro output should be treated as an adjustable estimate rather than a measured value.

Nutrition source consistency is another limitation. Five classes use verified supplemental sources because the available INDB sheet did not contain safe equivalent rows. This decision is also connected to the mapping strategy: it is safer to document a supplemental value than to force a weak fuzzy match into the database. Other classes may still use close substitutes, so confidence values and source metadata remain important for interpreting the output.

Dataset noise and class confusion also affect the result. Visual verification found only a small number of annotation-level issues, but object detectors are sensitive to label quality. The confusion matrix also shows that some visually similar classes are harder to separate. These limitations are expected in a 72-class food detector, especially when dishes share colour, texture, garnish, or serving style.

The recommendation layer is useful for presenting short dietary suggestions, but it depends on an external language model service when configured. The numerical macro calculation is rule-based and easier to verify. For a future version, replacing the language model dependency with a self-trained recommendation model would make the advice more consistent, controllable, and testable.

5.17 Safety and Ethics

Nutrition applications can influence user behaviour, so the interface should not present estimates as medical advice. Users with diabetes, kidney disease, hypertension, heart disease, or other health-sensitive conditions should consult qualified professionals before making diet decisions based on an automated tool. The application should also communicate uncertainty when model confidence is low, a serving estimate is approximate, or a nutrition value comes from a supplemental source.

Food images may reveal personal eating habits and health patterns. A production version should minimize image retention, protect API communication, and make user consent clear. The current prototype is local and does not implement cloud storage for uploaded images, but privacy would become more important if the system were deployed beyond a controlled project environment.

5.18 Lessons Learned

The project showed that applied AI systems require more than model selection. Dataset curation, naming conventions, mapping tables, API design, error handling, and UI workflow affected the final result as much as the detector choice. Food recognition is especially dependent on domain knowledge because visual labels and nutrition records rarely use identical names.

Another useful lesson was the value of validation artifacts. The class verification sheets, mapping coverage table, macro checks, prediction batches, and frontend workflow tests made it easier to find where a wrong output came from. These artifacts also made the report more evidence-based, because the claims can be traced back to files and checks rather than only screenshots.

Chapter 6

CONCLUSION

This project developed a full-stack food recognition and nutrition-assistance system for Indian meals. The implemented prototype brings together a YOLO-based detector, a curated 72-class dataset, a SQLite nutrition database, a FastAPI backend, and a Next.js frontend. In the working flow, a user uploads a food image, views detected foods with bounding boxes, checks macro values, adjusts serving assumptions, and receives dietary suggestions based on personal goals and health conditions.

The main objectives were achieved. Five YOLO-format source datasets were merged into a canonical Indian food dataset. The build process normalized raw labels, exported balanced splits, and produced machine-readable artifacts. Visual class verification confirmed that all 72 class names have corresponding image samples and that the class list is aligned. This mattered because the same class list had to support model training, nutrition mapping, and frontend display. The nutrition pipeline produced 1,010 food records, including five verified supplemental rows, and created 103 total YOLO-to-database mappings. For the expanded dataset, all 72 classes now have nutrition mappings.

The final YOLO11s detector was trained and evaluated on the expanded dataset. In `merge_results/`, the consolidated 100-epoch CSV recorded a best validation mAP@50 of 0.83379 and a best validation mAP@50:95 of 0.69226. The latest split-level image-evaluation curves reported 0.776 mAP@50 on validation images and 0.765 mAP@50 on the held-out test images. The backend and frontend were implemented as a working local prototype, not only as a model experiment. The backend includes image analysis, nutrition validation, macro calculation, goal, health-condition, and recommendation endpoints. The frontend handles image upload, detection visualization, macro display, serving adjustment, and recommendation review.

The work shows that food recognition becomes more useful when it is connected to nutrition databases and user-aware recommendation logic. Dataset normalization and nutrition mapping are not minor details in Indian food systems; they decide whether the output can be trusted. The prototype still has limitations in portion estimation, annotation quality, class confusion among similar dishes, and nutrition source consistency. Even with these limitations, it provides a working base for a controlled recommendation model, improved portion handling, mobile use, long-term tracking, and stronger safety review.

Chapter 7

FUTURE WORK

7.1 Self-Trained Recommendation Model

The current prototype uses a language model to generate natural-language dietary suggestions from detected foods, macro values, goals, and health conditions. In a future version, this dependency should be replaced with a self-trained recommendation model. That model can be trained on curated Indian meal patterns, nutrition rules, health-condition constraints, and expert-reviewed recommendation examples. This would make the recommendation layer more controllable, easier to validate, and less dependent on external LLM services.

The self-trained model should receive structured inputs from the existing backend, including detected food classes, calories, protein, carbohydrates, fat, serving estimates, user goal, calorie target, and health profile. Its output should be constrained to safe dietary suggestions and should include confidence or rule-trace information so that users can understand why a recommendation was produced. This would also make testing easier because the same input profile should produce consistent and reviewable advice.

7.2 Portion Size Estimation

The current system reports nutrition per 100 grams and allows serving adjustment. Future versions should estimate portion size using reference objects, depth estimation, segmentation masks, or plate-size assumptions. Instance segmentation could improve food-region estimation compared with bounding boxes, especially for rice, chutneys, and mixed curries. This would improve calorie usefulness because serving size is one of the largest sources of uncertainty in image-based food logging.

7.3 Mobile Deployment

A mobile version would make the system more practical for real meal logging. The frontend could be adapted as a progressive web app or native mobile application. Depending on device constraints, the model could run through server-side inference, quantized local inference, or a hybrid approach. Camera access, offline behaviour, and response time would need careful testing because users are more likely to log food from a phone than from a desktop system.

7.4 Personalized Long-Term Tracking

Future versions can add user accounts, meal history, daily calorie budgets, trend charts, and weekly nutrition summaries. This would move the prototype from a single-image analyzer toward a long-term dietary assistant. The design should still allow users to edit detected meals manually, because long-term tracking becomes useful only when users can correct mistakes and maintain accurate records.

7.5 Clinical Safety and Explainability

Before deployment for health-sensitive users, the system should include stronger disclaimers, uncertainty indicators, and rule-based safeguards. Recommendations for conditions such as diabetes, kidney disease, hypertension, and heart disease should be reviewed by qualified nutrition professionals. The interface should clearly separate general diet guidance from medical advice, especially when confidence is low or when the detected meal contains approximate nutrition mappings.

REFERENCES

[1] R. Agarwal, T. Choudhury, N. J. Ahuja, and T. Sarkar, “IndianFoodNet: Detecting indian food items using deep learning,” *International Journal of Computational Methods and Experimental Measurements*, vol. 11, no. 4, pp. 221–232, 2023.

[2] O. Prabhu, “Khana: A comprehensive Indian cuisine dataset,” *arXiv preprint arXiv:2509.06006*, 2025.

[3] “Enhanced food image recognition using transfer learning,” 2024, supplied literature review paper.

[4] K. Garisa, R. K. Kumar, and P. Singh, “Single-item training for multi-dish recognition: A class-agnostic framework for Indian food platters,” *Frontiers in Computer Science*, vol. 8, p. 1753764, 2026.

[5] R. U. Haque, R. H. Khan, A. S. M. Shihavuddin, M. M. M. Syeed, and M. F. Uddin, “Lightweight and parameter-optimized real-time food calorie estimation from images using CNN-based approach,” *Applied Sciences*, vol. 12, no. 19, p. 9733, 2022.

[6] Y. Arora, A. Arun, and C. V. Jawahar, “What is there in an Indian Thali?” in *Proceedings of the 16th Indian Conference on Computer Vision, Graphics and Image Processing*, 2025.

[7] A. Vijayakumar, H. B. Dubasi, A. Awasthi, and L. M. Jaacks, “Development of an Indian food composition database,” *Current Developments in Nutrition*, vol. 8, no. 7, p. 103790, 2024.

[8] National Institute of Nutrition, “Indian Food Composition Tables,” 2017, supplied literature review reference.

[9] “Synergistic frameworks for Indian food computing: An analysis of nutritional and visual dataset compatibility,” 2025.

[10] M. Al-Saffar and W. R. Baiee, "Nutrition information estimation from food photos using machine learning based on multiple datasets," *Bulletin of Electrical Engineering and Informatics*, vol. 11, no. 5, pp. 2922–2929, 2022.

[11] A. Jha, T. Gadekar, and S. Joshi, "Label AI: Barcode scanning based mapping of nutritional values to fitness planning," *International Journal of Computer Applications*, vol. 187, no. 54, pp. 60–68, 2025.

[12] A. M. Saad, M. R. H. Rahi, M. M. Islam, and G. Rabbani, "Diet engine: A real-time food nutrition assistant system for personalized dietary guidance," *Food Chemistry Advances*, vol. 7, p. 100978, 2025.

[13] S. Hussain, N. Khan, S. A. A. Shah, S. Bano, and A. W. Khan, "AI-driven personalized meal planning: A web-based platform for tailored nutrition and health management," *International Journal of Computer Applications*, vol. 187, no. 57, pp. 17–29, 2025.

[14] S. Khamesian, A. Arefeen, S. M. Carpenter, and H. Ghasemzadeh, "NutriGen: Personalized meal plan generator leveraging large language models to enhance dietary and nutritional adherence," *arXiv preprint arXiv:2502.20601*, 2025.

[15] M. Tokic, C. Livada, T. Galba, and A. Baumgartner, "Leveraging LLMs and computer vision for personalized nutrition advice," in *Engineering Proceedings*, vol. 125, no. 1, 2026, p. 21.

[16] "AI-based nutrition recommendation system," 2025, supplied literature review paper.

[17] C. O'Hara, G. Kent, A. C. Flynn, E. R. Gibney, and C. M. Timon, "An evaluation of ChatGPT for nutrient content estimation from meal photographs," *Nutrients*, vol. 17, no. 4, p. 607, 2025.

[18] "Visual nutrition analysis using food segmentation and nutrient regression," 2024, supplied literature review paper.

[19] Roboflow Universe Contributor, "Indian food detection dataset, version 1," https://universe.roboflow.com/project-z0tql/indian-food-detection-kzw9g/dataset/1, 2023, roboflow Universe export; YOLOv11 format; CC BY 4.0; local metadata exported on 28 December 2024.

[20] ——, "indian food dataset, version 6," https://universe.roboflow.com/microplastics-vypxl/indian-food-txofy/dataset/6, 2025, roboflow Universe export; YOLOv11 format; CC BY 4.0; local metadata exported on 1 December 2025.

[21] R. Agarwal, N. Bansal, T. Sarkar, T. Choudhury, and N. J. Ahuja, "IndianFood-7 dataset, version 2," https://universe.roboflow.com/indianfood/indian_food-pwzlc/dataset/2, 2024, roboflow Universe export; YOLOv11 format; CC BY 4.0; local metadata exported on 24 October 2024.

[22] ——, "IndianFoodNet-30 dataset, version 1," https://universe.roboflow.com/indianfoodnet/indianfoodnet/dataset/1, 2023, roboflow Universe export; YOLOv8 format; CC BY 4.0; local metadata exported on 20 April 2023.

[23] Roboflow Universe Contributor, "south indian food detection dataset, version 19," https://universe.roboflow.com/bharani-1ucid/south-indian-food-detection/dataset/19, 2024, roboflow Universe export; YOLOv11 format; CC BY 4.0; local metadata exported on 24 October 2024.

[24] FastAPI Project, "FastAPI documentation," https://fastapi.tiangolo.com/, 2024, software documentation. Accessed: 6 June 2026.

[25] Vercel, "Next.js documentation," https://nextjs.org/docs, 2024, software documentation. Accessed: 6 June 2026.

[26] "Evolve bhakarwadi nutrition facts per 100 g," https://www.fatsecret.co.in/calories-nutrition/evolve/bhakarwadi/100g, 2026, FatSecret India. Accessed: 6 June 2026.

[27] "Ghevar recipe calories and nutrition per 100 g," https://clearcals.com/recipes/ghevar/, 2026, Clearcals. Accessed: 6 June 2026.

[28] "Jalebi recipe calories and nutrition per 100 g," https://clearcals.com/recipes/jalebi/, 2026, Clearcals. Accessed: 6 June 2026.

[29] "Khandvi recipe calories and nutrition per 100 g," https://clearcals.com/recipes/khandvi/, 2026, Clearcals. Accessed: 6 June 2026.

[30] "Nandu Kari recipe calories and nutrition per 100 g," https://clearcals.com/recipes/nandu-kari/, 2026, Clearcals. Accessed: 6 June 2026.

Appendix A

DATASET CLASSES AND NUTRITION MAPPINGS

A.1 Expanded Dataset Class List

The final dataset contains 72 canonical classes. The class list is stored in the generated dataset configuration file, and visual verification confirmed that every class has image samples. The mapping table and per-class count table below include the complete class set so that the training dataset can be checked directly from the report.

A.2 Class-to-INDB Mapping Table

Table A.1 lists the database food name used for each dataset class. Most rows come from INDB. The few classes absent from the available INDB sheet use verified supplemental rows, as noted in the project database. The table is included here so that the nutrition assumptions are visible instead of remaining only inside the generated SQLite file.

Table A.1: Full 72-class dataset to INDB mapping table

Dataset class	Matched database food name	Score
aloo_gobi	Potato cauliflower (Aloo gobhi)	1.00
aloo_masala	Potato curry (Aloo ki sabzi)	0.90
appam	Appam	1.00
beetroot_poriyal	Vegetables stir fry	0.75

Dataset class	Matched database food name	Score
besan_cheela	Gram flour chilla/cheela (Besan chilla/cheela)	1.00
bhakarwadi	Bhakarwadi	1.00
bhakri	Chapati/Roti	0.80
bhatura	Bhatura	1.00
bhindi_masala	Okra/Lady’s fingers fry (Bhindi sabzi/sabji/subji)	0.95
biryani	Vegetable biryani/biriyani	0.85
carrot_poriyal	Carrot and cabbage with coconut (Nariyal ke saath pattagobhi aur gajar)	0.85
chai	Hot tea (Garam Chai)	1.00
chicken	Chicken curry	0.85
chicken_65	Chilli chicken	0.80
chicken_biryani	Chicken pulao	0.80
chole	Chickpeas curry (Safed channa curry)	1.00
coconut_chutney	Coconut chutney (Nariyal ki chutney)	1.00
dal	Mixed dal	0.90
dhokla	Dhokla	1.00
dosa	Plain dosa	0.95
dum_aloo	Dum aloo	1.00
eggs	Boiled egg (Ubla anda)	0.95
fish_curry	Fish curry (Machli curry)	1.00
ghevar	Ghevar	1.00
green_chutney	Green chutney	1.00
gulab_jamun	Gulab Jamun with khoya	1.00
idli	Idli	1.00
jalebi	Jalebi	1.00
kaara_chutney	Tomato chutney (Tamatar ki chutney)	0.85
kali	Maize porridge	0.70

Dataset class	Matched database food name	Score
kebab	Boti kebab	0.90
khandvi	Khandvi	1.00
kheer	Rice kheer (Chawal ki kheer)	1.00
koozh	Maize porridge	0.70
kulfi	Kulfi	1.00
lassi	Sweet Lassi (Meethi lassi)	0.95
lemon_rice	Lemon rice (Pulihora, Elumichai sadam, Chitranna)	1.00
medu_vada	Plain urad dal vada (Uzunne vada/Minapa garelu/Ulundu vadai/Medu vada)	1.00
modak	Semolina ladoo with coconut (Suji/Rava aur nariyal ke ladoo)	0.70
mushroom_biryani	Mushroom pulao	0.80
mutton_biryani	Mutton biryani/biriyani	1.00
mutton_curry	Mutton korma	0.85
nandu_masala	Nandu Kari (Crab masala)	1.00
nei_satham	Plain pulao	0.80
omelette	Plain omelette/omlet	1.00
onion_pakoda	Onion pakora/pakoda (Pyaaz ke pakode)	1.00
paal_kolukattai	Rice kheer (Chawal ki kheer)	0.75
palak_paneer	Spinach paneer (Palak paneer)	1.00
paneer_biryani	Paneer pulao	0.80
paratha	Plain parantha/paratha	0.95
parupu_vadai	Fermented bengal gram vada (Khameerikrit/Ufna hua channa dal ka vada)	0.90
pidi_kolukattai	Rice puttu (Ari puttu)	0.75
poha	Poha	1.00
poorna_kolukattai	Semolina ladoo with coconut (Suji/Rava aur nariyal ke ladoo)	0.70
prawn_thokku	Prawn curry (with coconut) (Jhinga curry)	0.85

Dataset class	Matched database food name	Score
puri	Poori	1.00
raita	Cucumber raita (Kheere ka raita)	0.90
rajma_curry	Kidney bean curry (Rajmah curry)	1.00
ras_malai	Rasmalai	1.00
rice	Boiled rice (Uble chawal)	1.00
roti	Chapati/Roti	1.00
saag	Sarson ka saag	1.00
salad	Tossed salad	0.90
sambar	Sambar	1.00
sambar_satham	Vegetable khichdi/khichri	0.80
samosa	Potato samosa (Aloo ka samosa)	1.00
shahi_paneer	Shahi paneer	1.00
thepla	Methi thepla	0.95
upma	Semolina upma (Suji/Rava upma)	0.95
veg_briyani	Vegetable biryani/biriyani	1.00
veg_pulao	Mixed vegetable pulao	1.00
ven_pongal	Plain khitchdi (Plain khichri/khichdi)	0.75

A.3 Known Annotation Issues

Table A.2: Sample-level annotation issues found during visual verification

File	Finding
train27229-jalebi.jpg	Labeled as `jalebi`, but the image does not visually look like jalebi.
train18550-kaara-chutney label	`kaara_chutney` box is placed on the vada instead of the visible chutney.
train32747-nandu-masala label	Full image is crab, but the bounding box is extremely thin and produces a blank crop.

A.4 Visual Verification Sheet Set

The following figures reproduce the complete visual verification sheet set. The original eight contact sheets were divided into top and bottom halves, producing sixteen larger half-sheets for clearer reading in the printed report. The sheets were generated from actual dataset files, with sampled full images and label-based crops used to check whether class names matched the visible food items. These sheets helped during review because they gave a quick visual check of the class list without opening the dataset folder manually.

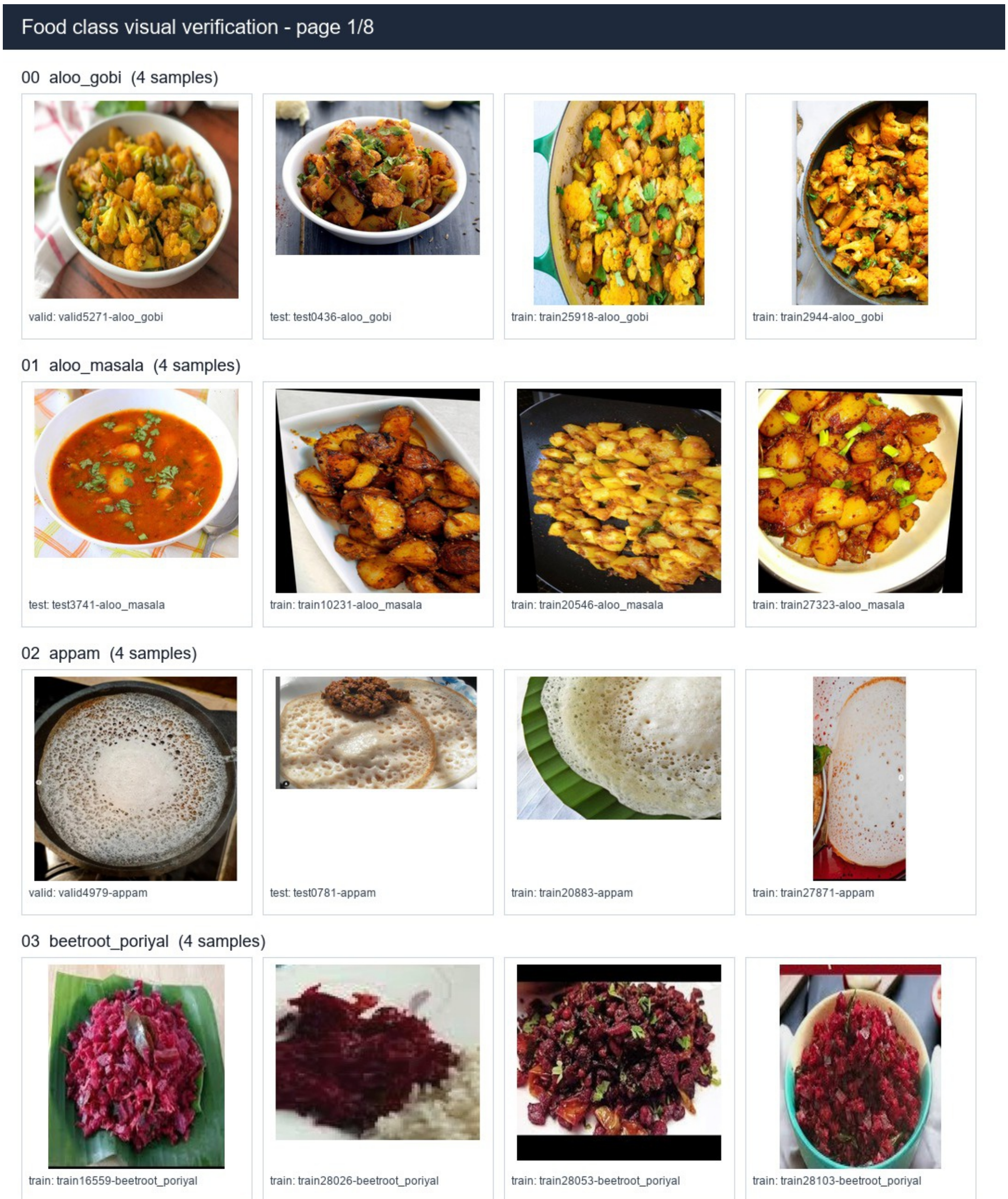


Figure A.1: Visual verification half-sheet 1

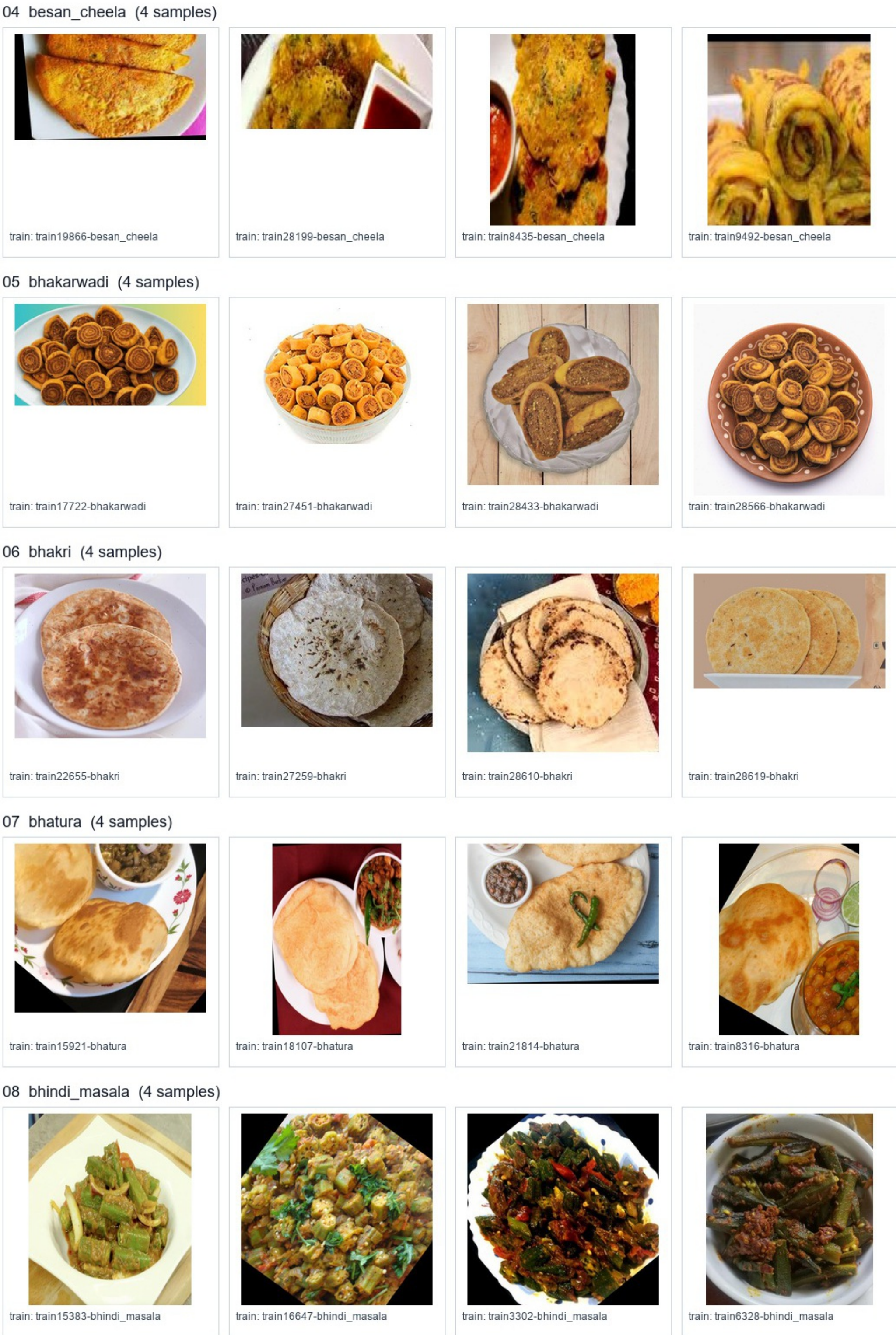


Figure A.2: Visual verification half-sheet 2

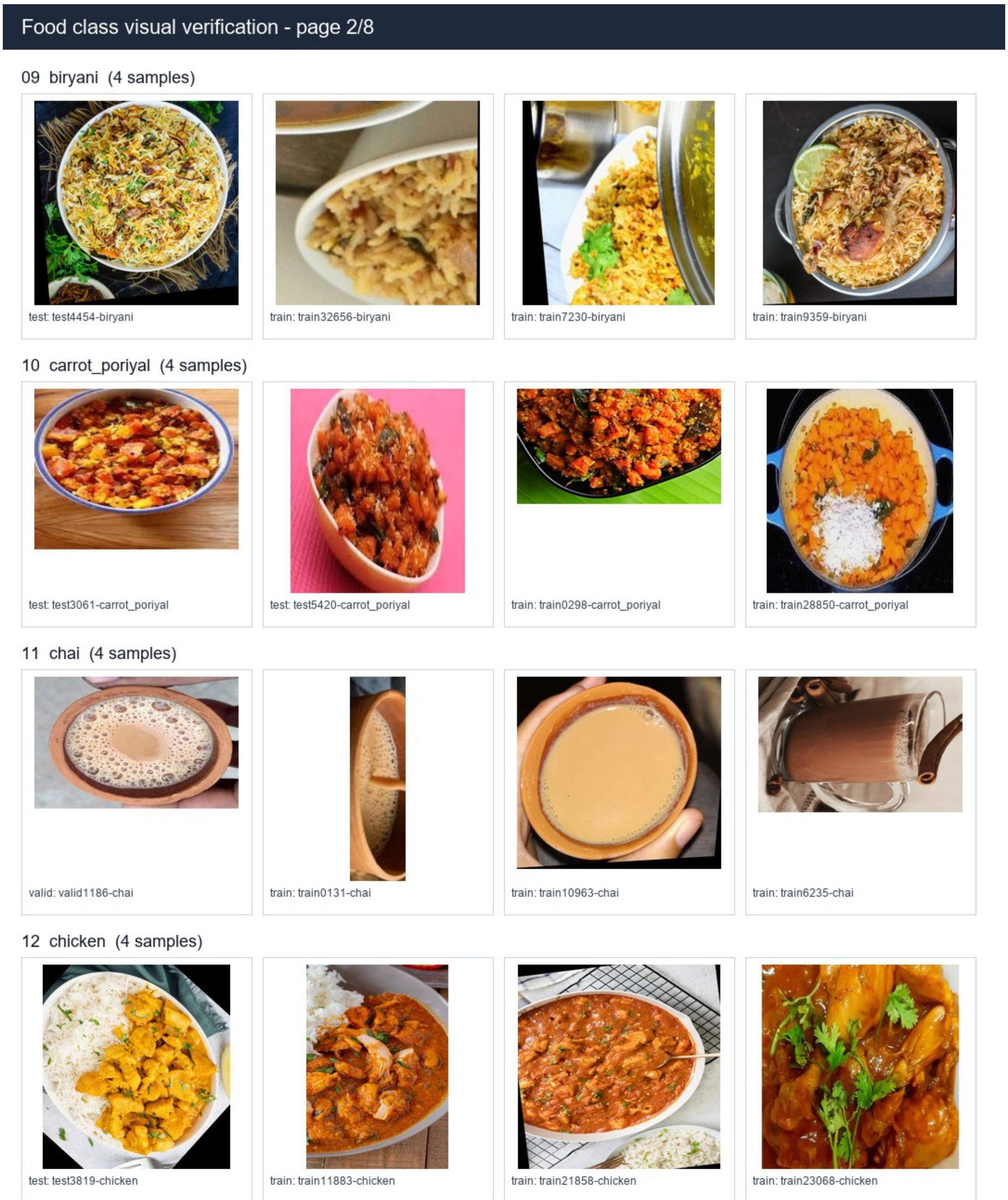


Figure A.3: Visual verification half-sheet 3

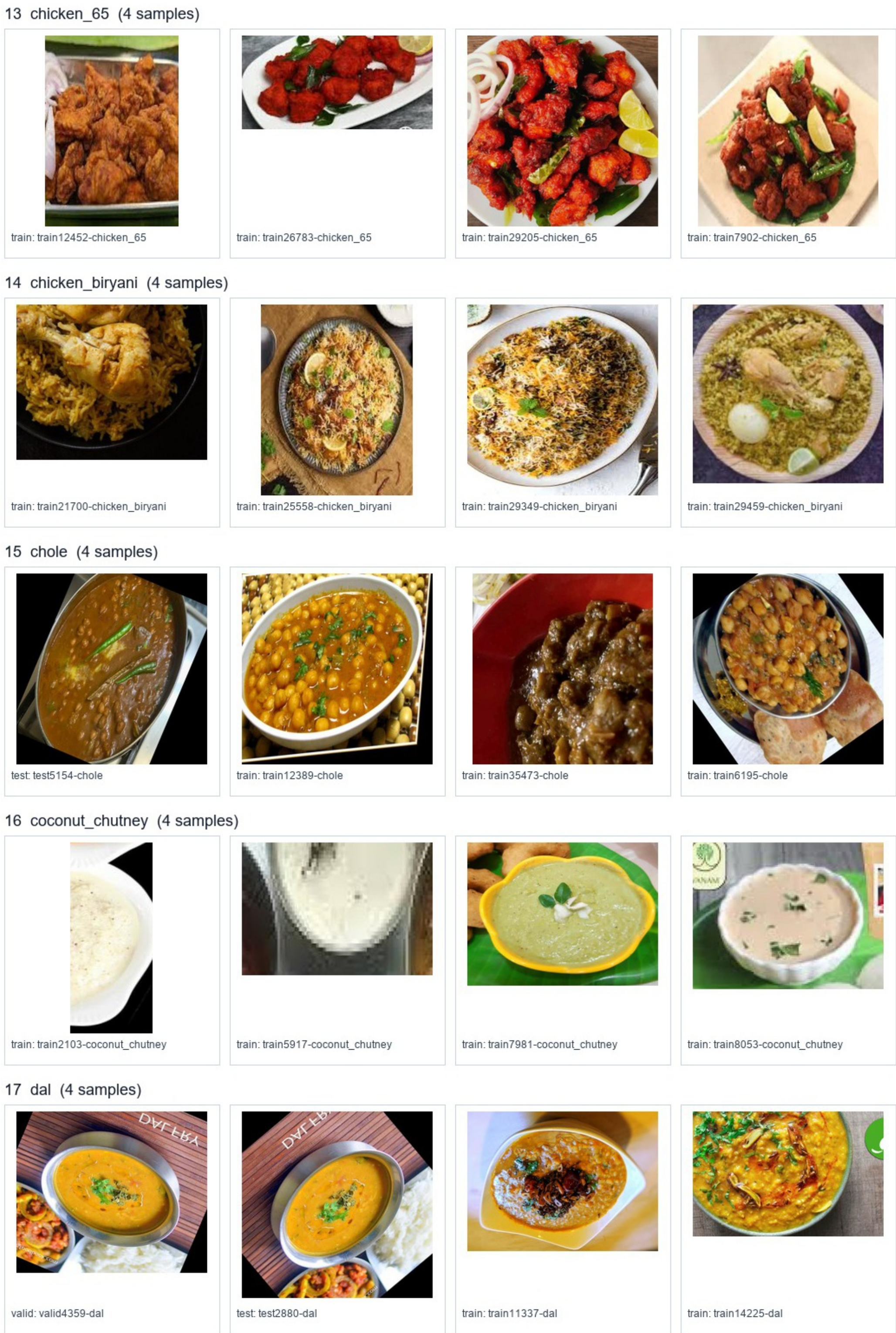


Figure A.4: Visual verification half-sheet 4

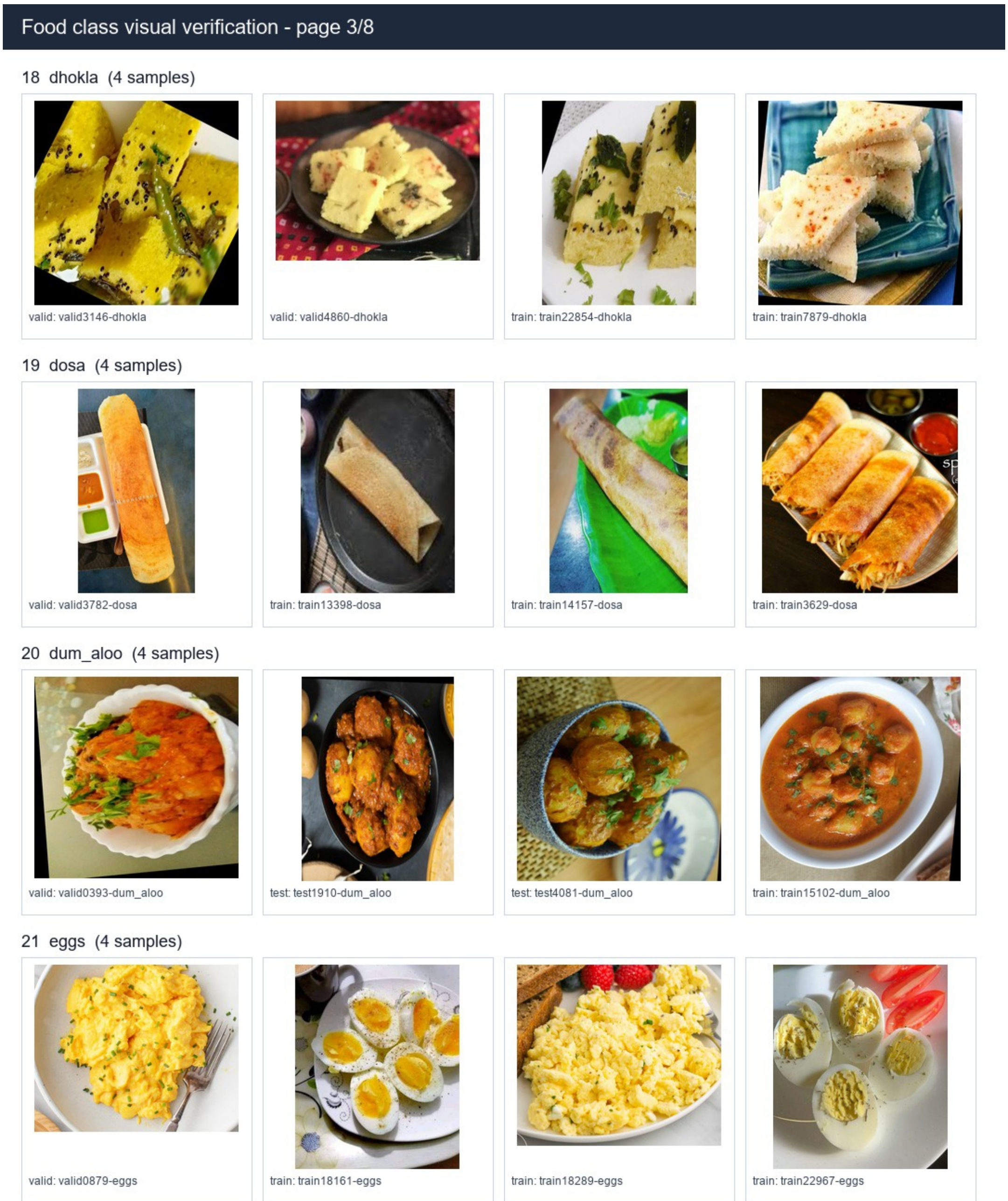


Figure A.5: Visual verification half-sheet 5

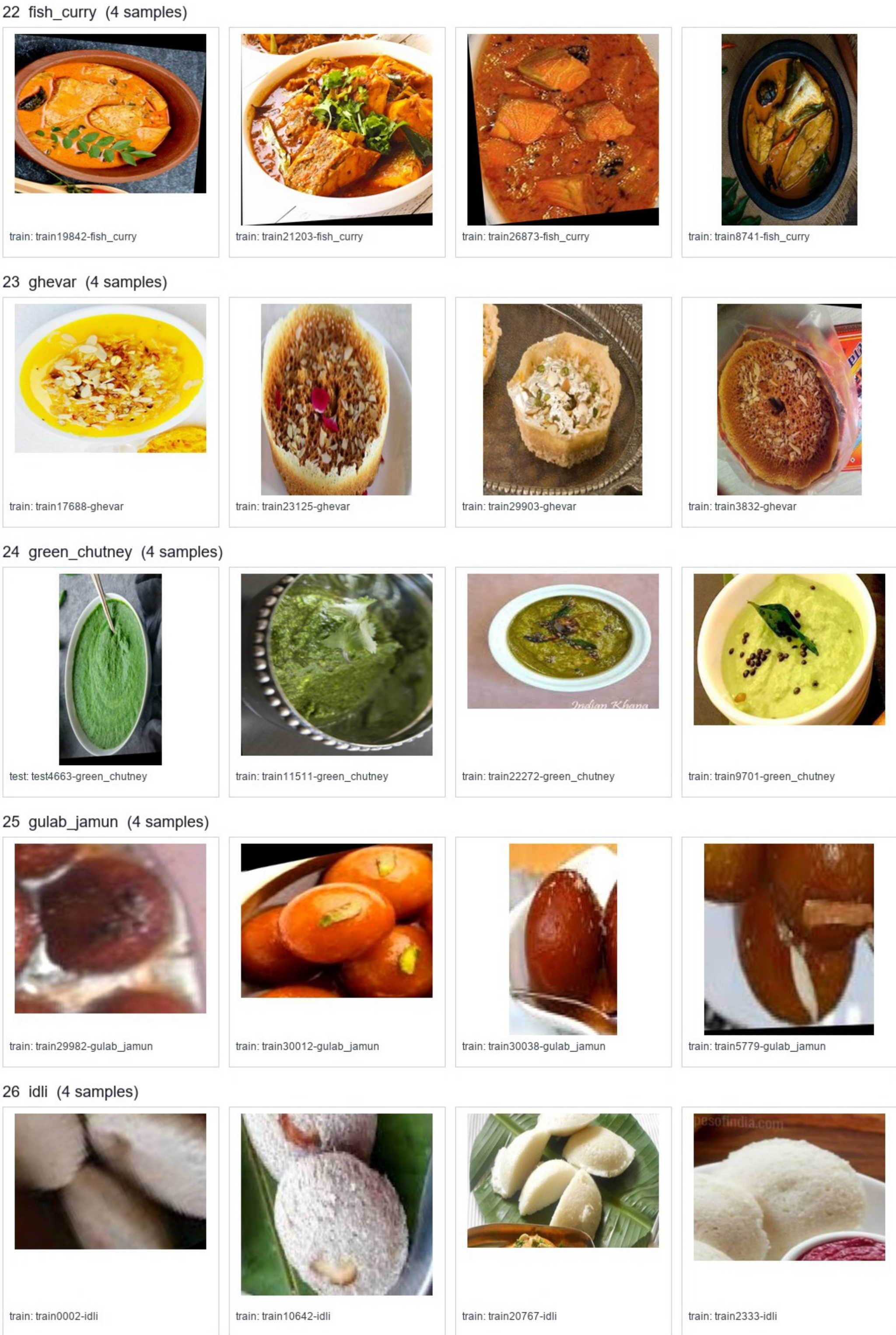


Figure A.6: Visual verification half-sheet 6

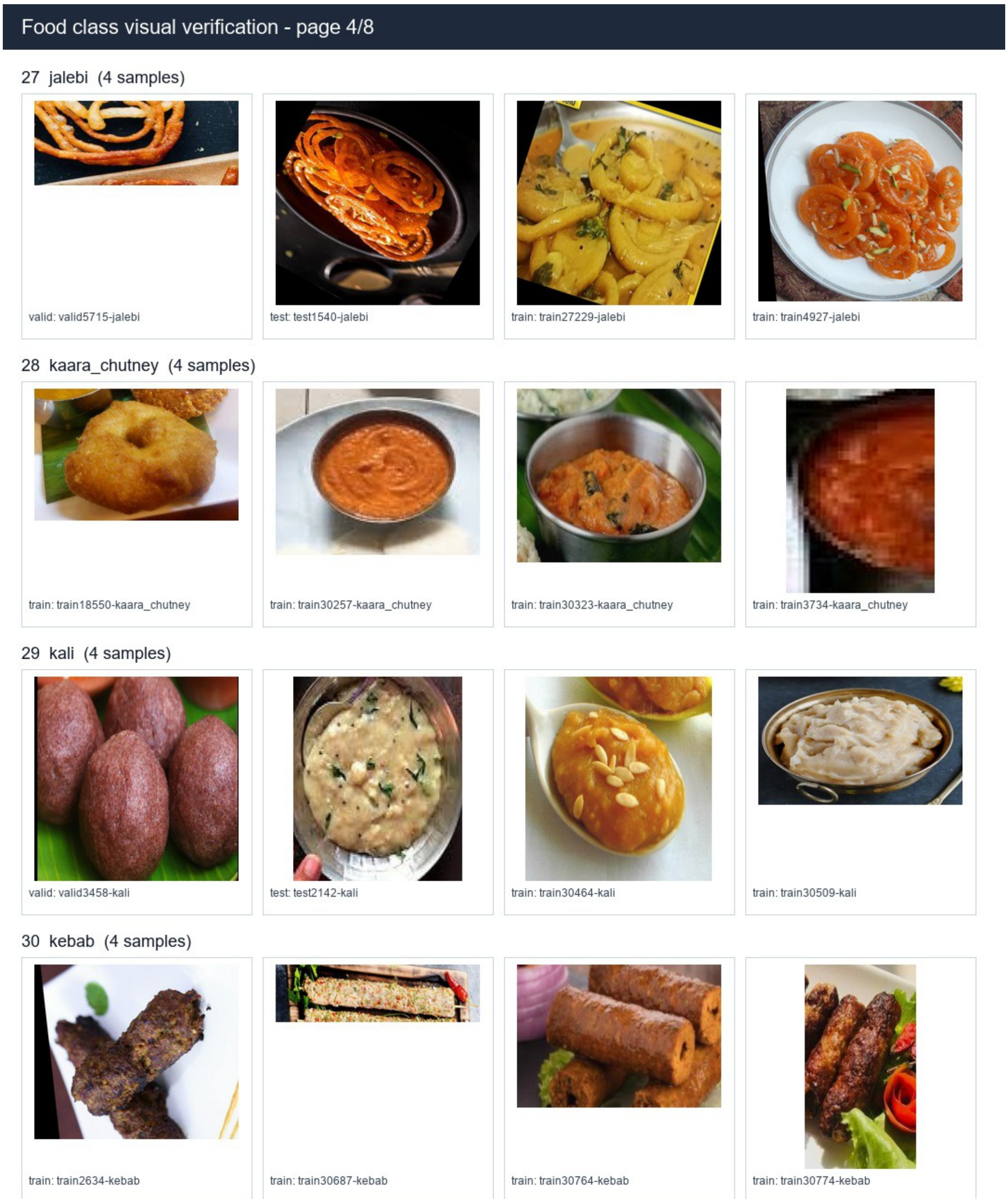


Figure A.7: Visual verification half-sheet 7

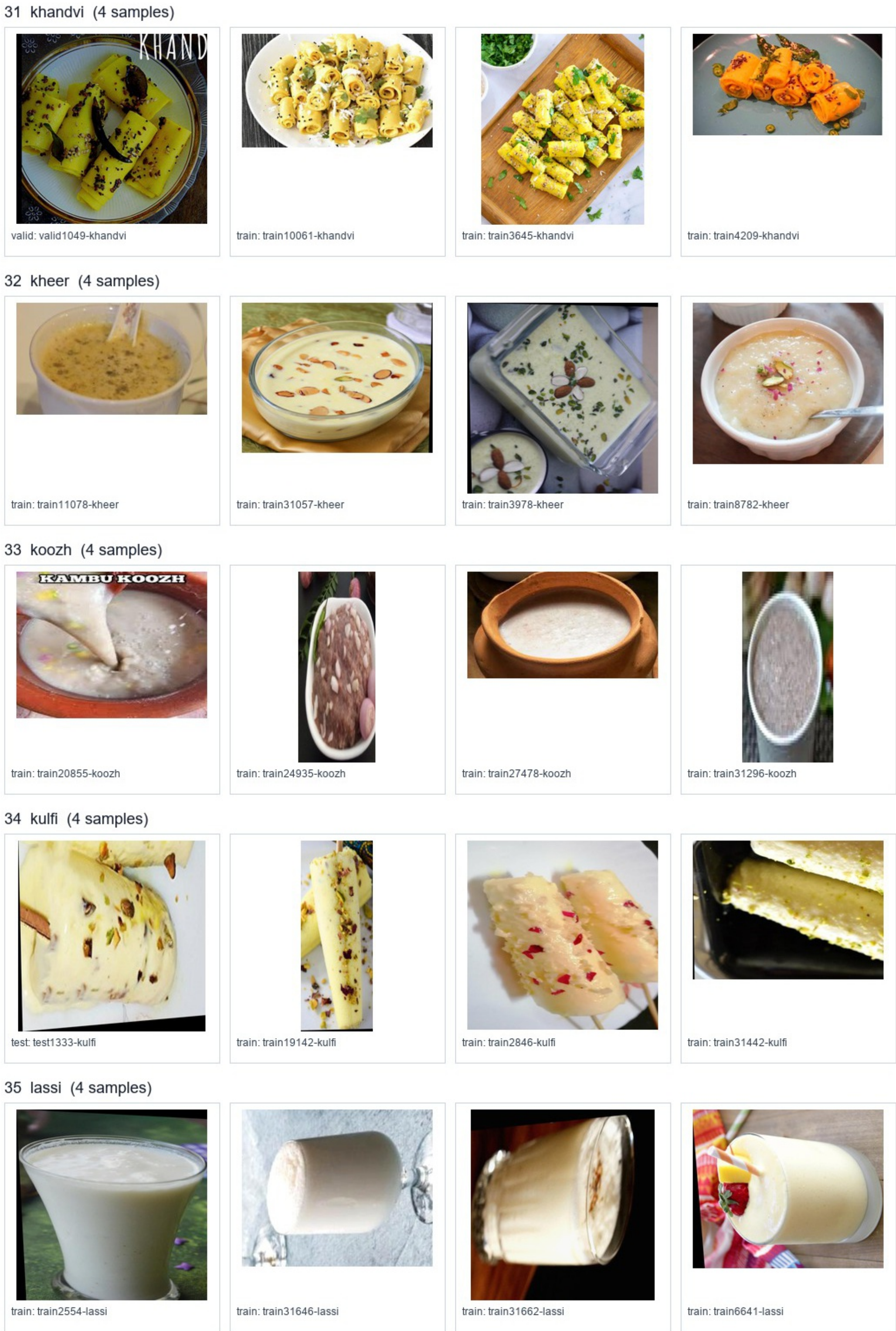


Figure A.8: Visual verification half-sheet 8

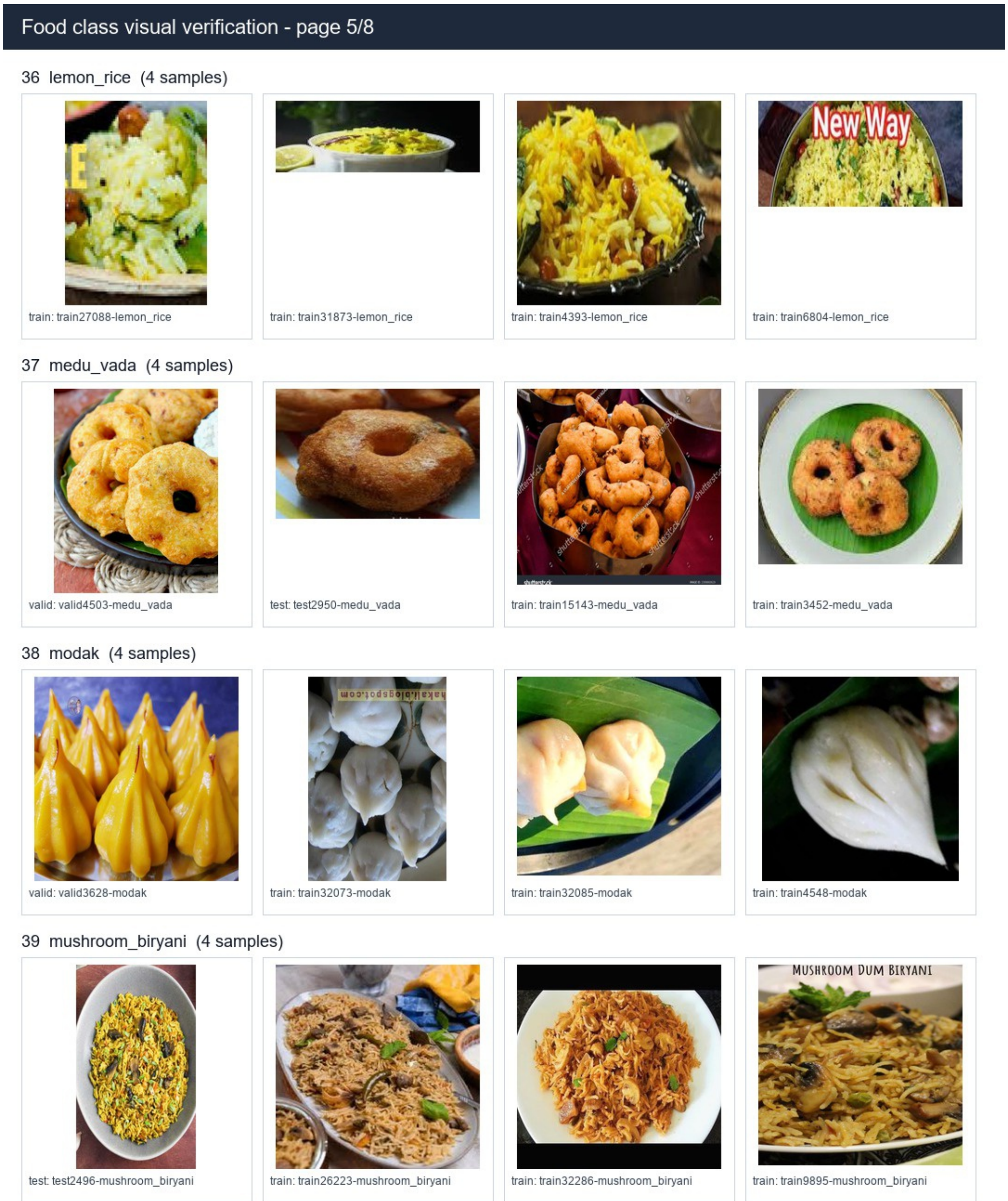


Figure A.9: Visual verification half-sheet 9

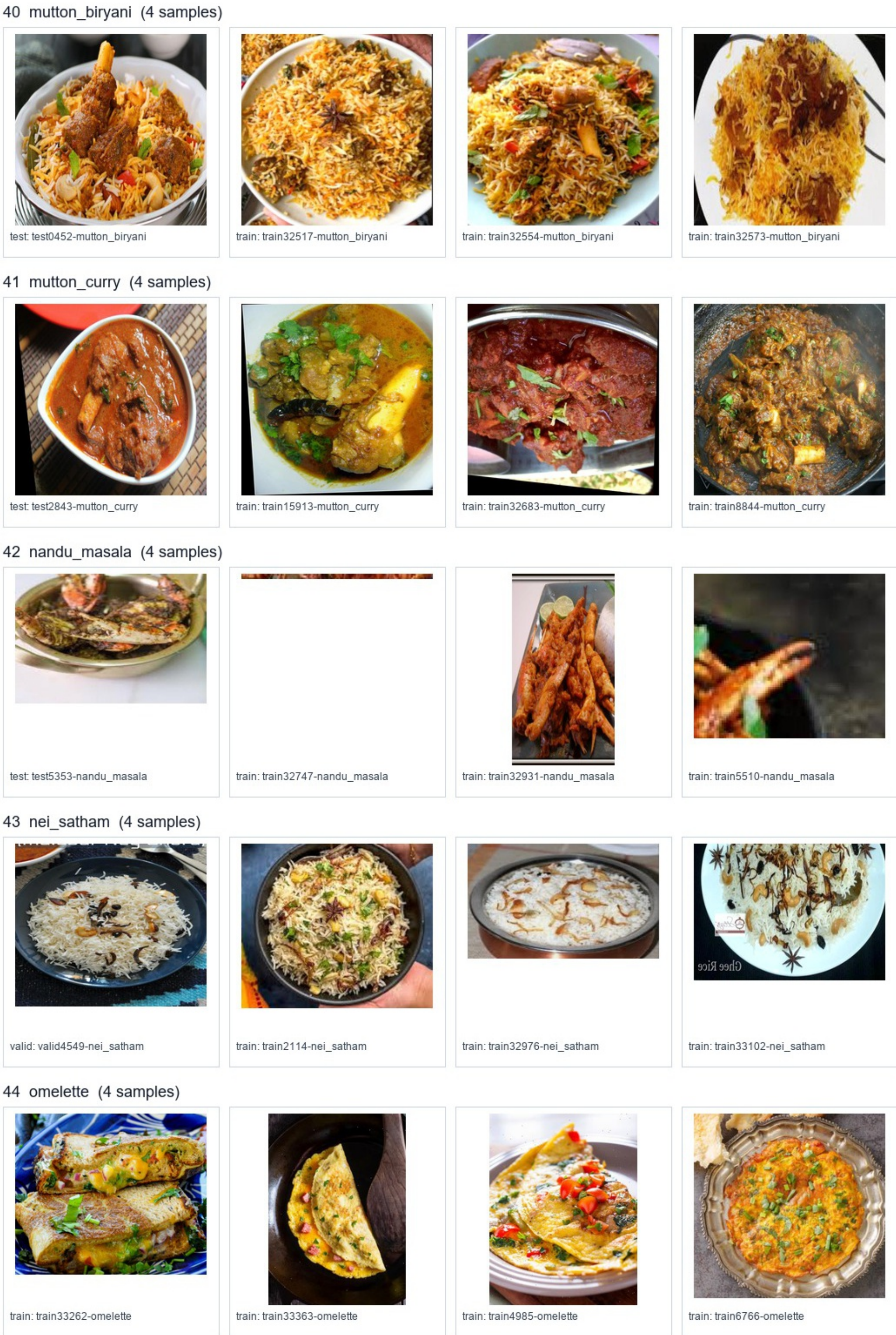


Figure A.10: Visual verification half-sheet 10

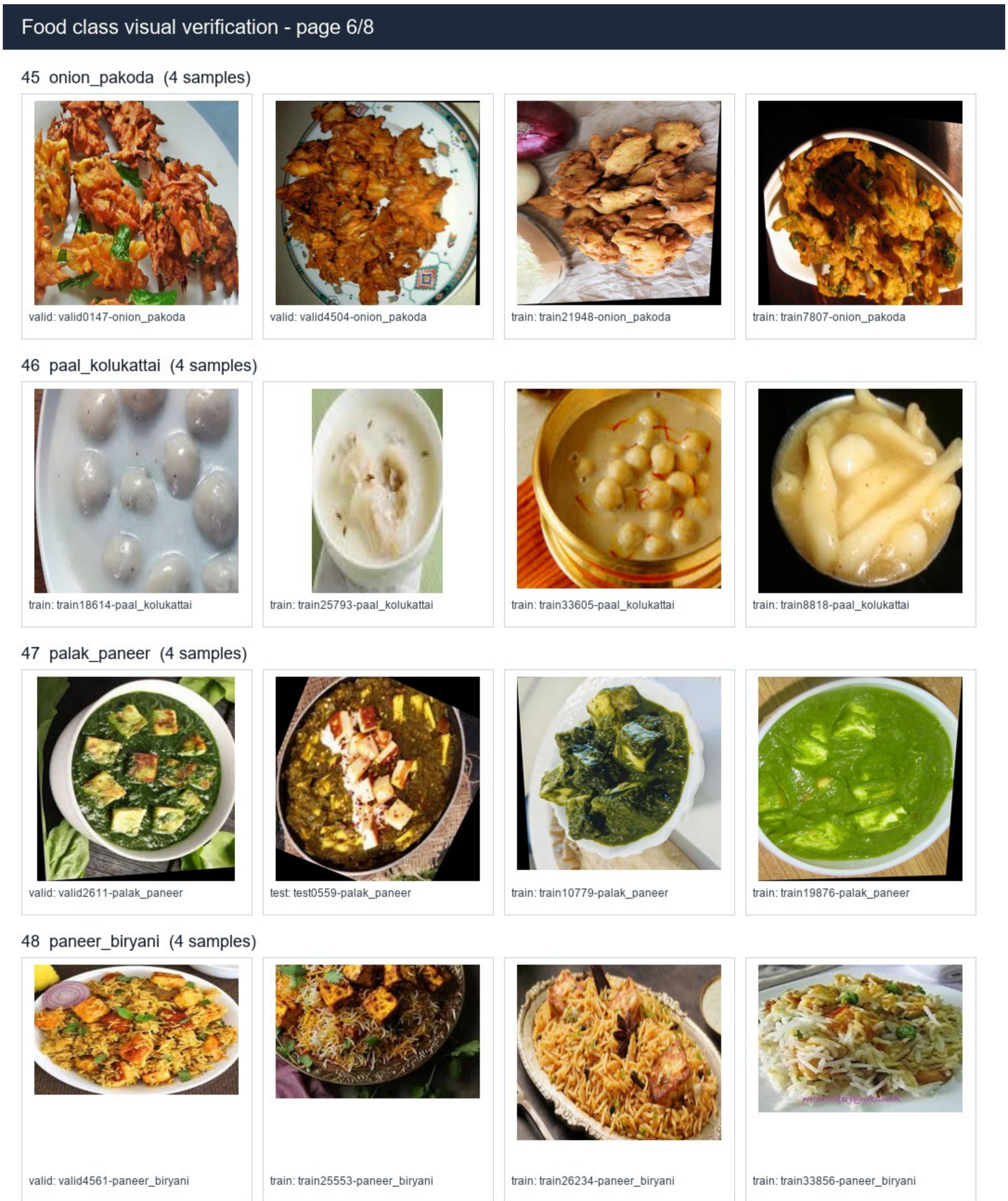


Figure A.11: Visual verification half-sheet 11

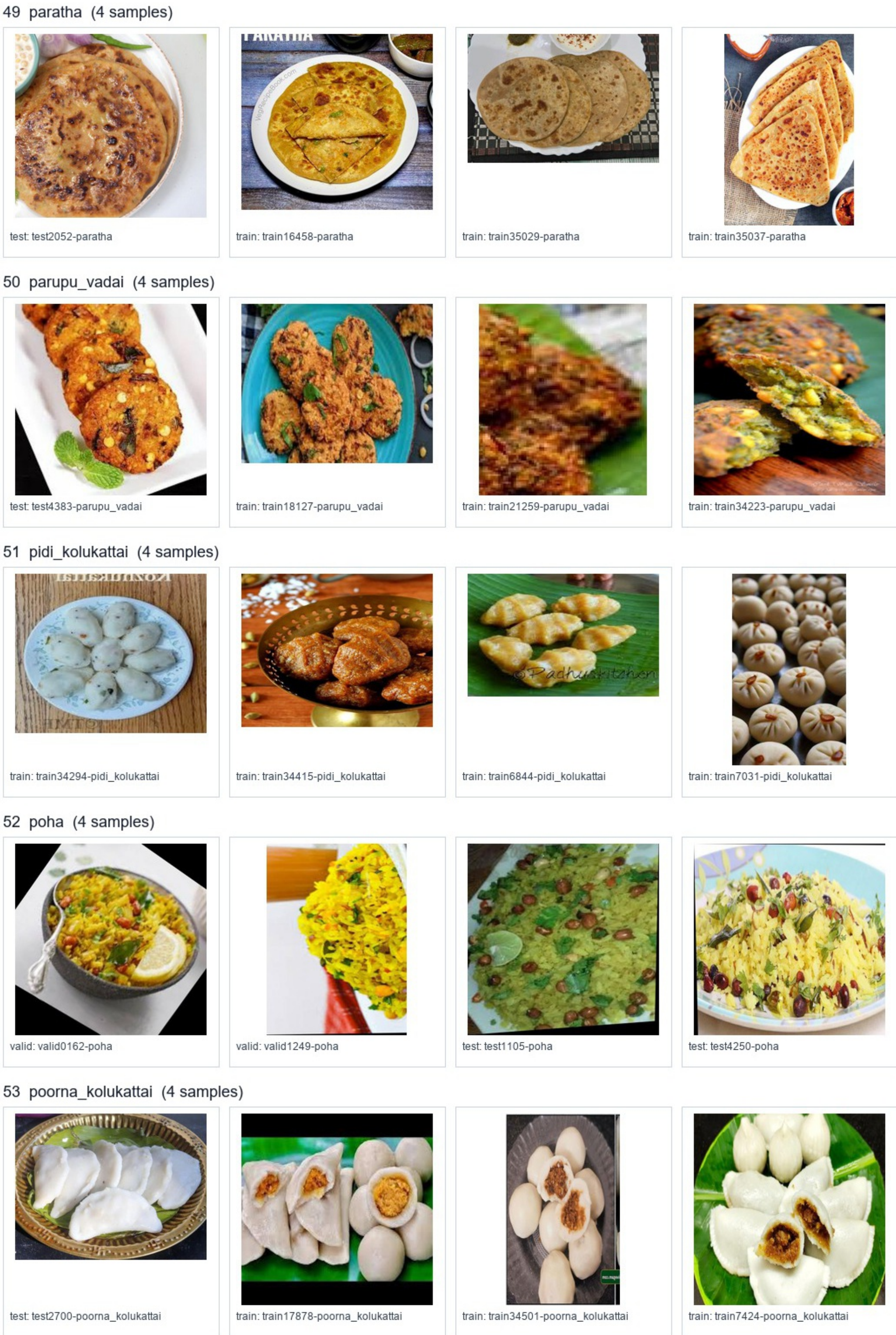


Figure A.12: Visual verification half-sheet 12

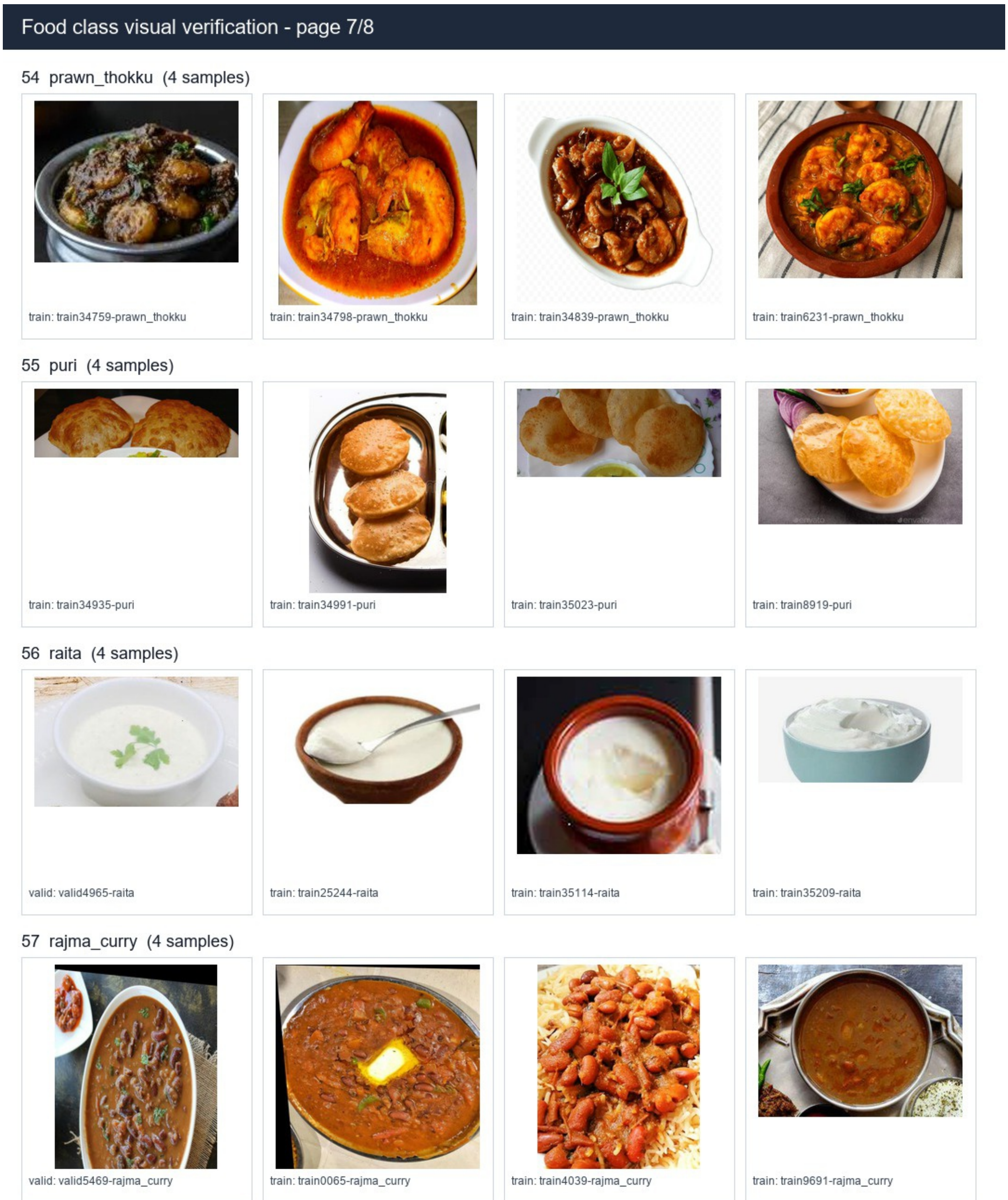


Figure A.13: Visual verification half-sheet 13

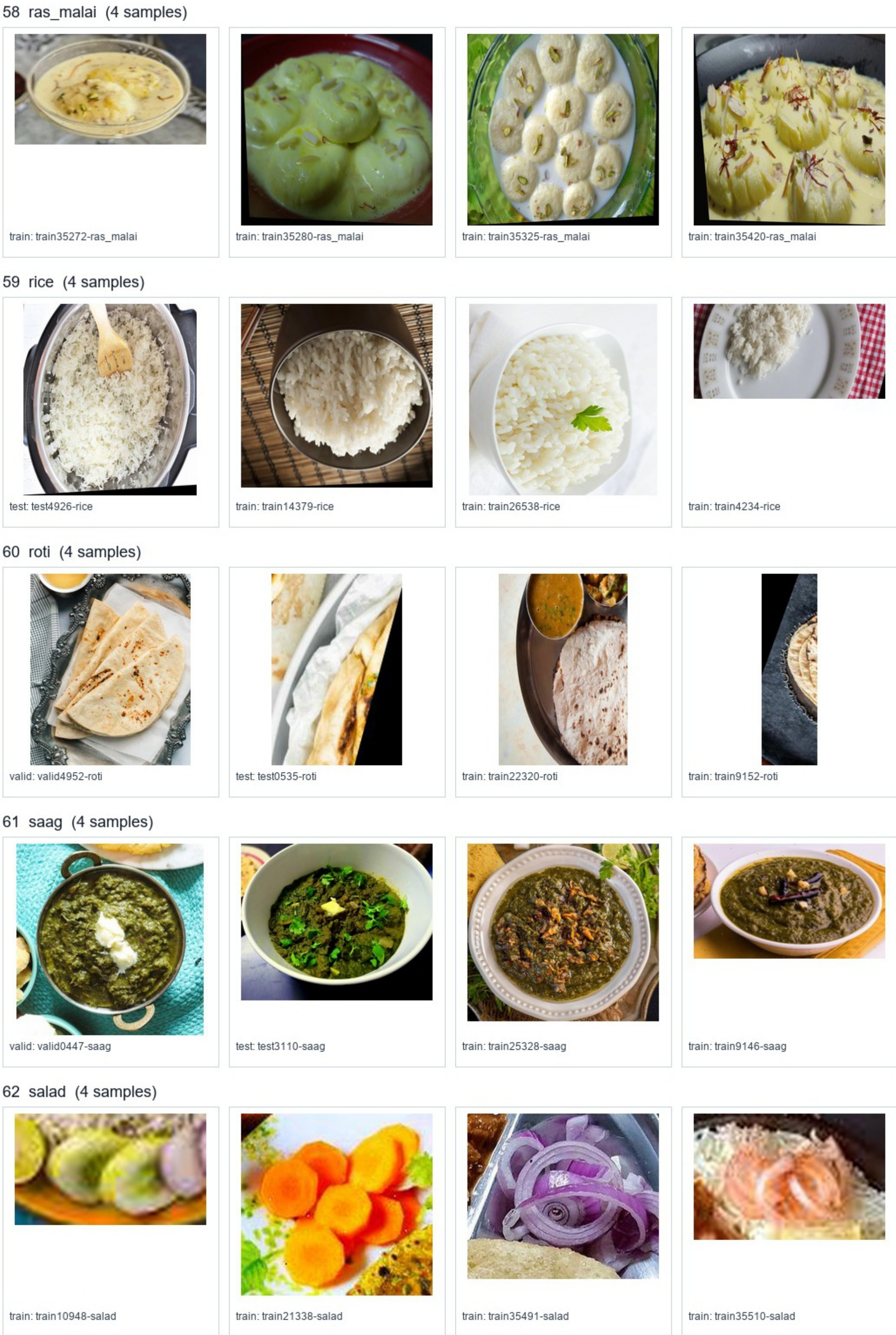


Figure A.14: Visual verification half-sheet 14

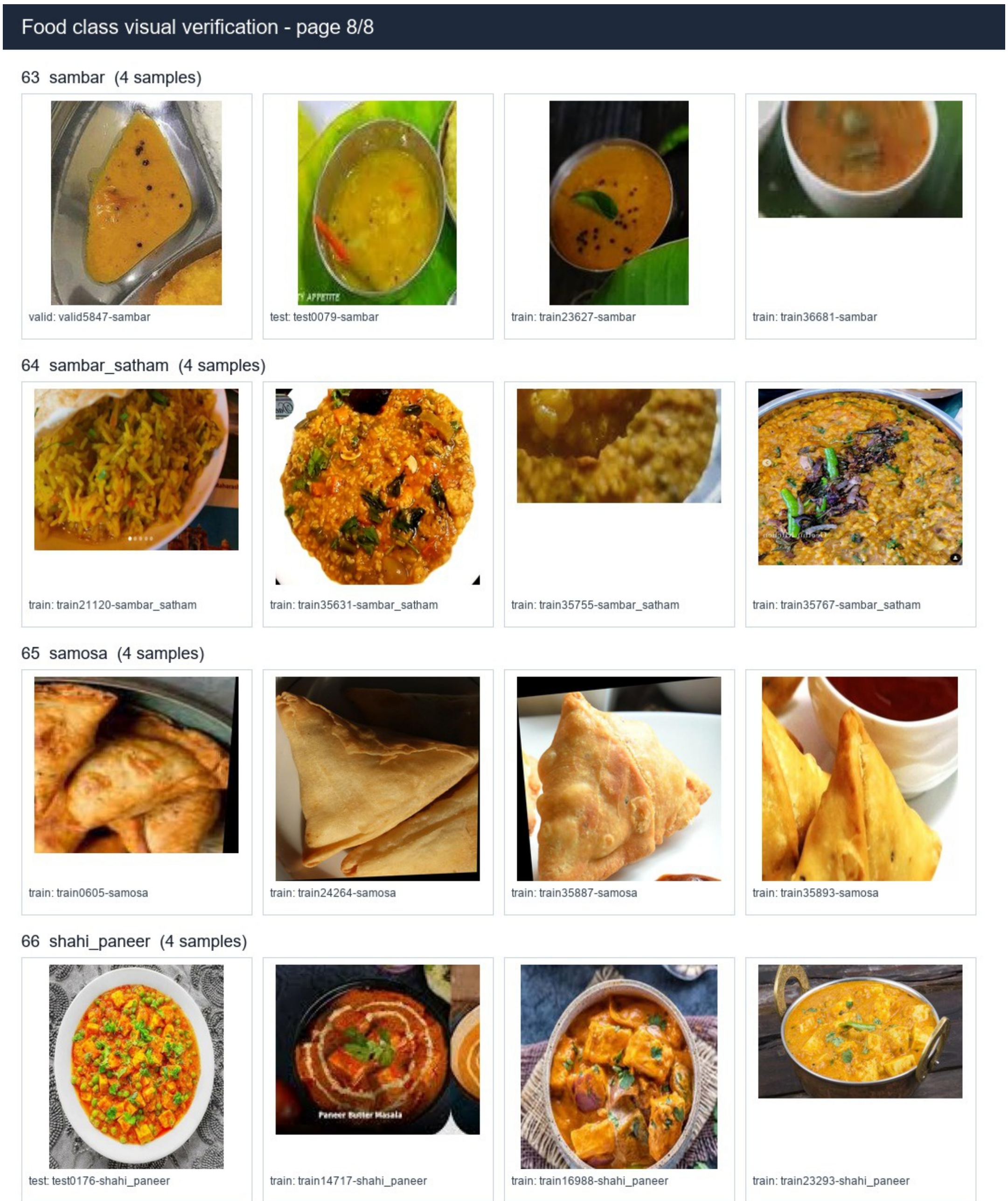


Figure A.15: Visual verification half-sheet 15

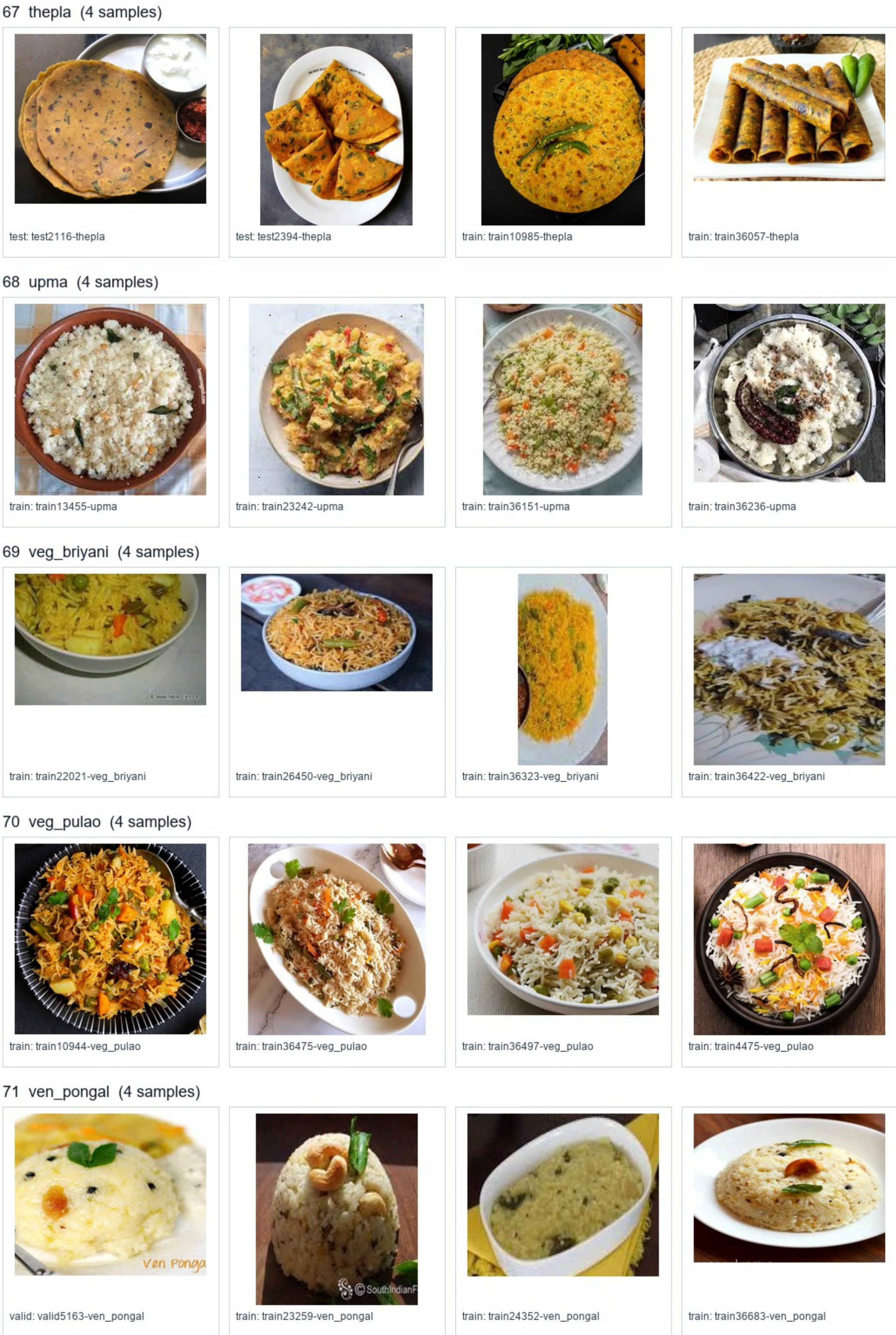


Figure A.16: Visual verification half-sheet 16

A.5 Per-Class Dataset Counts

Table A.3: Per-class dataset counts after merge and train augmentation

Class	Train before → after	Valid	Test
aloo_gobi	556 → 568	119	119
aloo_masala	520 → 528	111	112
appam	210 → 421	45	45
beetroot_poriyal	210 → 421	45	45
besan_cheela	207 → 414	44	45
bhakarwadi	256 → 500	55	55
bhakri	79 → 168	17	17
bhatura	680 → 680	145	146
bhindi_masala	1046 → 1056	224	224
biryani	700 → 703	150	150
carrot_poriyal	210 → 421	45	45
chai	340 → 511	73	73
chicken	925 → 925	198	198
chicken_65	210 → 421	45	45
chicken_biryani	210 → 420	45	45
chole	1277 → 1362	273	274
coconut_chutney	1007 → 1207	216	216
dal	970 → 985	208	208
dhokla	703 → 704	151	151
dosa	1125 → 1227	241	241
dum_aloo	358 → 500	77	77
eggs	519 → 561	111	111
fish_curry	362 → 500	77	78
ghevar	235 → 470	50	51
green_chutney	661 → 748	141	142
gulab_jamun	272 → 501	58	59
idli	821 → 889	176	176

Class	Train before → after	Valid	Test
jalebi	872 → 872	187	187
kaara_chutney	218 → 457	46	47
kali	210 → 423	45	45
kebab	161 → 322	34	35
khandvi	325 → 500	70	70
kheer	294 → 500	63	63
koozh	210 → 421	45	45
kulfi	332 → 500	71	71
lassi	322 → 500	69	69
lemon_rice	210 → 420	45	45
medu_vada	386 → 522	83	83
modak	120 → 240	26	26
mushroom_biryani	210 → 420	45	45
mutton_biryani	210 → 421	45	45
mutton_curry	350 → 500	75	75
nandu_masala	210 → 420	45	45
nei_satham	210 → 420	45	45
omelette	212 → 440	45	46
onion_pakoda	333 → 500	71	72
paal_kolukattai	210 → 420	45	45
palak_paneer	1000 → 1000	214	214
paneer_biryani	210 → 420	45	45
paratha	127 → 282	27	28
parupu_vadai	210 → 425	45	45
pidi_kolukattai	210 → 422	45	45
poha	1049 → 1058	224	225
poorna_kolukattai	210 → 421	45	45
prawn_thokku	210 → 420	45	45
puri	380 → 535	81	82

Class	Train before → after	Valid	Test
raita	239 → 554	51	52
rajma_curry	1008 → 1050	216	216
ras_malai	322 → 500	69	69
rice	1776 → 1856	380	381
roti	803 → 821	172	172
saag	525 → 525	112	113
salad	157 → 360	34	34
sambar	475 → 617	102	102
sambar_satham	210 → 420	45	45
samosa	358 → 500	77	77
shahi_paneer	878 → 878	188	189
thepla	174 → 377	37	38
upma	145 → 290	31	31
veg_briyani	210 → 420	45	45
veg_pulao	170 → 364	36	37
ven_pongal	210 → 433	45	45

Appendix B

API RESPONSE EXAMPLES

B.1 Analyze Food Response

The following listing shows the simplified structure of a successful image analysis response. The backend response follows this structure so that the frontend can render detection boxes, nutrition cards, and macro totals from predictable fields.

Listing B.1: Simplified food analysis response


```
{
  "success": true,
  "food_detected": true,
  "image_width": 760,
  "image_height": 787,
  "detections": [
    {
      "food_label": "Chole",
      "display_name": "Chickpeas curry (Safed channa curry)",
      "confidence": 0.95,
      "bbox": {
        "x1": 442,
        "y1": 194,
        "x2": 697,
        "y2": 551
      },
      "macros": {
        "calories": 163.4,
        "protein_g": 6.1,
        "carbs_g": 20.0,
```

```
      "fat_g": 6.8
    },
    "nutrition_source": "INDB"
  }
 ]
}
```


B.2 Macro Calculation Request

Listing B.2: Macro calculation request


```
{
 "goal": "weight_loss",
 "target_calories": 2000,
 "health_condition": "diabetic"
}
```


B.3 Macro Calculation Response

Listing B.3: Macro calculation response


```
{
 "goal": "weight_loss",
 "target_calories": 2000,
 "health_condition": "diabetic",
 "macros": {
   "carbs_percent": 33,
   "protein_percent": 41,
   "fat_percent": 26,
   "carbs_g": 165,
   "protein_g": 205,
   "fat_g": 58
 }
}
```

B.4 No Food Response

Listing B.4: No-food response shape

```
{
  "success": true,
  "food_detected": false,
  "food_not_found": true,
  "detections": [],
  "message": "No food items detected in the uploaded image."
}
```

B.5 Macro Target Reference Tables

The following tables show the macro targets generated for a 2,000 kcal daily target. The backend normalizes goal and health-condition modifiers before converting calories to grams. These tables are included as reference outputs so that the macro calculation logic can be checked without running the application.

Table B.1: Weight-loss macro targets for 2,000 kcal

Health profile	Carb %	Protein %	Fat %	Carb g	Protein g	Fat g
None	40	35	25	200	175	56
Diabetic	33	41	26	165	205	58
Hypertension	41	36	23	205	180	51
Heart disease	43	36	21	215	180	47
High cholesterol	42	39	19	210	195	42
Digestive issues	43	34	23	215	170	51
Kidney disease	50	23	27	250	115	60
Anemia	40	38	22	200	190	49
Thyroid disorder	39	36	25	195	180	56

Table B.2: Muscle-gain macro targets for 2,000 kcal

Health profile	Carb %	Protein %	Fat %	Carb g	Protein g	Fat g
None	40	30	30	200	150	67
Diabetic	33	36	31	165	180	69
Hypertension	41	31	28	205	155	62
Heart disease	44	31	25	220	155	56
High cholesterol	43	34	23	215	170	51
Digestive issues	43	29	28	215	145	62
Kidney disease	49	19	32	245	95	71
Anemia	40	33	27	200	165	60
Thyroid disorder	39	31	30	195	155	67

Table B.3: Maintenance macro targets for 2,000 kcal

Health profile	Carb %	Protein %	Fat %	Carb g	Protein g	Fat g
None	50	25	25	250	125	56
Diabetic	43	31	26	215	155	58
Hypertension	51	26	23	255	130	51
Heart disease	54	26	20	270	130	44
High cholesterol	53	28	19	265	140	42
Digestive issues	53	24	23	265	120	51
Kidney disease	59	15	26	295	75	58
Anemia	50	28	22	250	140	49
Thyroid disorder	49	26	25	245	130	56

Table B.4: Endurance macro targets for 2,000 kcal

Health profile	Carb %	Protein %	Fat %	Carb g	Protein g	Fat g
None	55	20	20	275	100	44
Diabetic	51	26	23	255	130	51
Hypertension	59	22	19	295	110	42
Heart disease	62	21	17	310	105	38
High cholesterol	61	23	16	305	115	36
Digestive issues	61	20	19	305	100	42
Kidney disease	66	13	21	330	65	47
Anemia	58	23	19	290	115	42
Thyroid disorder	57	22	21	285	110	47